AAS | IOP Astronomy

Chapter 5

Stellar Structure and Evolution

> How often have I said to you that when you have eliminated the impossible, whatever remains, however improbable, must be the truth?
>
> Sir Arthur Conan Doyle

Stars evolve. From the moment of birth, a star's internal structure and appearance change with time. Some of these changes occur very slowly and hardly affect the local environment, but others are swift and have far-reaching repercussions. In this chapter, we review the basic evolution of stars and show how relatively simple stellar-evolution calculations can be conducted.

5.1 In a Nutshell

Stellar evolution is a very different game from Newtonian gravity. Whereas the physical equations underlying gravitational dynamics can be described and solved using basically high-school mathematics, solving the equations for stellar structure and evolution is challenging even for experts. In this chapter, we present an overview of stellar structure and evolution, although it will barely be enough to give an appreciation of the complexities of stellar-evolution codes. We start by reviewing some fundamental timescales in stellar evolution, followed by a rudimentary review of the structure equations and some prescriptions for stellar evolution. More extensive discussion can be found in Kippenhahn & Weigert (1967) and Eggleton (2006).

A star is a gravity-confined nuclear fusion reactor (Bethe 1939). Stars are born when parts of a molecular cloud are compressed by turbulence or other external forces (such as supernovae), collapse under their own weight, and fragment into a collection of prestellar clumps. These clumps continue to contract and accrete, and their central temperatures increase. Some of the excess heat is radiated away, but some of it goes to increase the pressure, ultimately halting the gravitational contraction. These hydrodynamical

doi:10.1088/2514-3433/adfc3cch5

processes are discussed in more detail in Chapter 6. A clump becomes a star when its central temperature and density reaches the point where nuclear fusion can start. This moment is generally associated with the birth of the star. At this point, the star is said to lie on the *zero-age main sequence* (ZAMS). From then on, the star evolves, as nuclear fusion in its core[1] drives irreversible structural change.

Figure 5.1 shows images of two red supergiant stars, Antares (Ohnaka et al. 2017) and Betelgeuse (Haubois et al. 2009), showing their complex surface temperature structure. Such observations have only recently become possible. By now, apart from the Sun (see Figure 5.5), a total of 26 stars have been resolved. A more theoretically oriented view of the internal structure of three stars of different masses is presented in Figure 5.2.

During its lifetime, a star experiences several evolutionary phases, each associated with distinct burning regimes in the central nuclear furnace. This evolution is often presented as a trajectory in temperature–luminosity space, in what is called a Hertzsprung–Russell (H–R) diagram. Figure 5.3 is an H–R diagram illustrating the evolution of three stars of different masses. Colors indicate various stellar-evolutionary stages. Very roughly speaking, we can divide the evolution of a star into three distinct phases: the main sequence, the post-main sequence (or giant phase), and the remnant phase, when the star has turned into a compact object. Along the main sequence, stars are generally comparable in size to the Sun. Giants can be much larger—as large as several astronomical units (see Figure 5.1). Remnants are much smaller than the Sun —comparable to or much smaller than Earth.

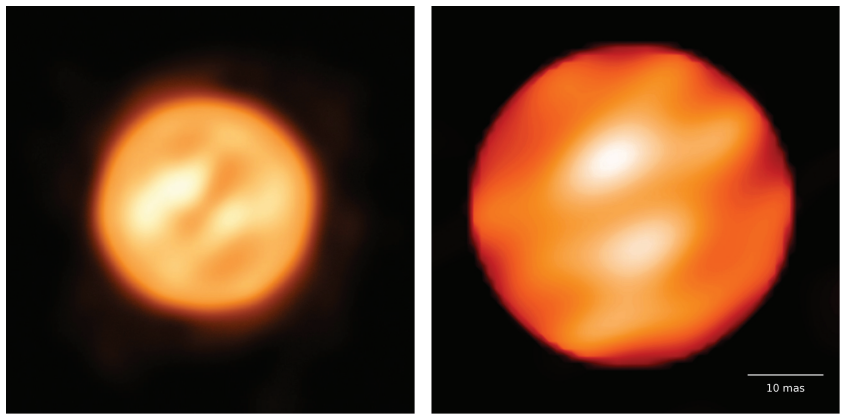

Figure 5.1. Stars are finite objects with structure on their surface, as demonstrated here with the observations of Antares (left, image with the ESA VLTI; image credit: ESA/Ohnaka et al. 2017) and Betelgeuse (right, image by Observatoire de Paris; image credit: NASA/Haubois et al. 2009). Both stars are similar in size—about 4.1–5.5 au (Dolan et al. 2016).

[1] The stellar core is the central portion of the star where nuclear fusion occurs. The transition from fusion to nonfusion is typically quite sharp; see Figure 5.4.

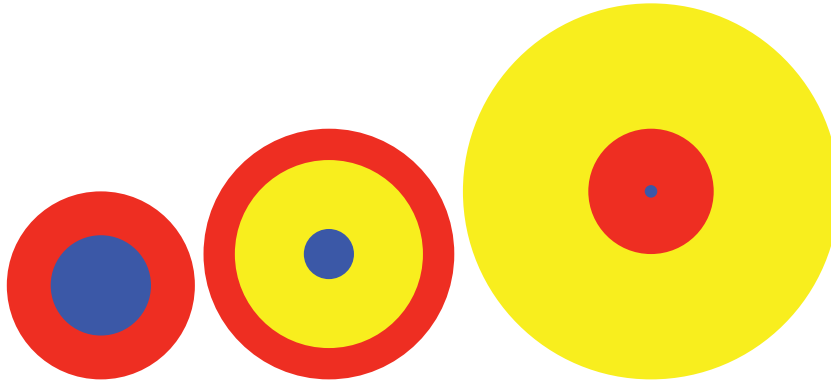

Figure 5.2. Schematic structure of a low-mass star (left), an intermediate-mass star (middle), and a high-mass star (right). They are not to scale, but the various colored regions in the stellar interior indicate the nuclear-burning region (blue), the convective zone (red), and the radiation-dominated region (yellow).

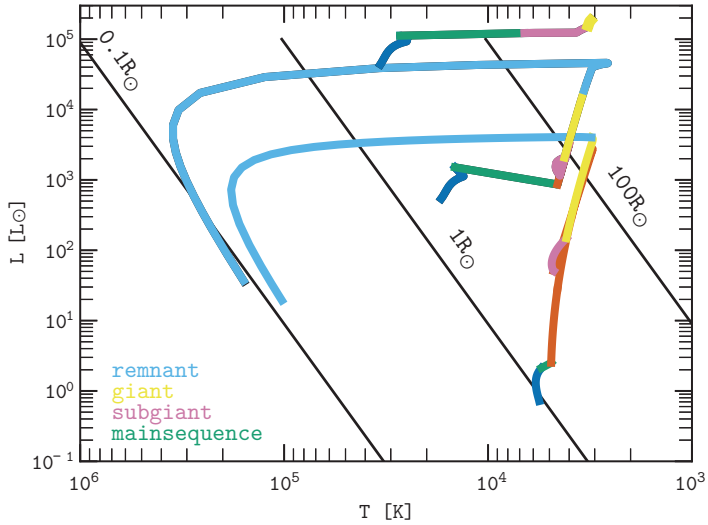

Figure 5.3. H–R diagrams (luminosity versus temperature) for three stars of masses 1 $M_\odot$ (bottom set curves), 5 $M_\odot$, and 20 $M_\odot$ (top curves), from the ZAMS to the moments they turn into compact remnants. The colored text in the bottom left identifies the evolutionary stage of the star along its track (with the caveat that a star at the tip of the asymptotic giant branch should probably not be identified as a remnant). The black diagonal lines indicate the run of luminosity and temperature for blackbodies with radii 0.01 $R_\odot$ (left), 1 $R_\odot$, and 100 $R_\odot$ (rightmost line). The figure was made using the script `amuse.examples.plot_stellar_evolution_track`.

To guide the eye in Figure 5.3, we plot lines of constant stellar radius r. The radius can be estimated from the star's effective temperature ($T_{\rm eff} \approx T$) and luminosity by approximating the stellar spectrum as a blackbody, resulting in

$$L = 4\pi r^2 \sigma T_{\rm eff}^4, \tag{5.1}$$

where σ is the Stefan–Boltzmann constant. This is a considerable simplification because the stellar spectrum is generally not described particularly well as a blackbody, but it is a reasonable first approximation to the true state of the star.

5.1.1 Stellar Timescales

Three fundamental timescales are relevant for stellar evolution. These are the *dynamical timescale* $t_{\rm dyn}$, the *Kelvin–Helmholtz timescale* $t_{\rm KH}$, and the *nuclear timescale* $t_{\rm nuc}$ (Bradt 2014). They are timescales on which distinctly different physical processes operate.

5.1.1.1 The Dynamical Timescale

The stellar analog to the dynamical timescale is defined analogously to the gravitational dynamical timescale discussed in Section 4.1.2. For a star of mass m and radius r, the dynamical timescale is

$$t_{\rm dyn} = \sqrt{\frac{r^3}{Gm}}. \tag{5.2}$$

This is the dynamical (orbital) timescale for a particle orbiting at the stellar surface, but it is also (approximately) the time taken for a sound wave to cross the star. The dynamical timescale for the Sun (`m = 1|units.MSun` and `r = 1|units.RSun`) can be calculated as follows:

```
import numpy as np
from amuse.units import units
from amuse.units import constants

def dynamical_time_scale(mass, radius, G=constants.G):
    return np.sqrt(radius**3/(G * mass))

print(
    f"solar dynamical time scale = "
    f"{dynamical_time_scale(1 | units.MSun, 1 | units.RSun).in_(units.minute)}"
)
```

which turns out to be about $t_{\rm dyn} \simeq 27$ minutes. If a comet hits the Sun, the other side of the star learns about the impact in about half an hour.

The related *convective timescale* $t_{\rm conv}$ is the timescale on which material convectively bubbles up from the stellar interior toward the surface. In convection, the stellar luminosity is carried outward by physical motion of the stellar gas rather than by radiation. It can be characterized by the mean distance Δr traveled by a typical rising gas parcel (usually assumed to be related to the local pressure scale height $P/|\nabla P|$) and the mean speed $\overline{v}$ with which the parcel rises:

$$t_{\rm conv} = \frac{\Delta r}{\overline{v}}. \tag{5.3}$$

For the Sun, $\Delta r \simeq 0.1 {\rm R}_\odot$ and $\overline{v} \simeq 11$ m s^{-1}. We then obtain $t_{\rm conv} \simeq 0.2$ years.

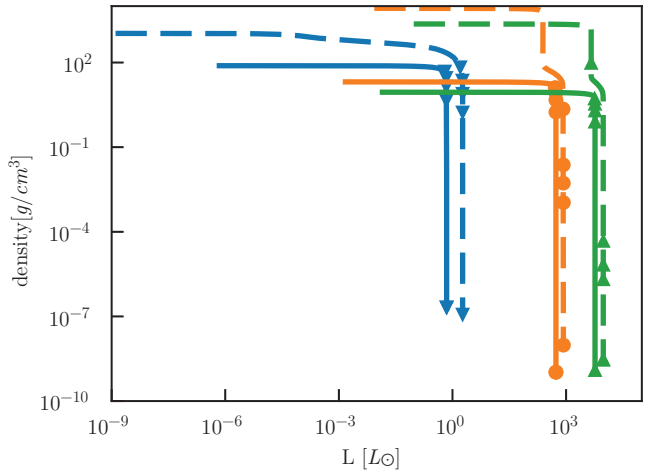

Figure 5.4. Density as a function of total luminosity in the stellar interior, showing the difference between the stellar core and envelope. Results are shown for three stellar masses: 1 $M_\odot$ (blue), 5 $M_\odot$ (red), and 10 $M_\odot$ (yellow). The solid curves are for stellar ages of 1 Myr; the dashed curves are for 11 Gyr, 90 Myr, and 20 Myr, for the 1 $M_\odot$, the 5 $M_\odot$, and 10 $M_\odot$ stars, respectively. Bullets and triangles are drawn at 20% intervals of the enclosed mass of the star. The script that produced this figure is `amuse.examples.plot_stellar_structure`. It should take a couple of minutes to run on a laptop.

5.1.1.2 The Thermal Timescale

The Kelvin–Helmholtz (or thermal) timescale is the timescale on which thermal energy leaves a star. It can be written as

$$t_{\rm KH} = \frac{E_{\rm th}}{L}, \tag{5.4}$$

where $E_{\rm th}$ is the star's total thermal energy and L is its luminosity, the total energy leaving the stellar surface per unit time.

Like star clusters, stars in equilibrium (i.e., having no bulk motion) also are governed by the virial theorem [see Equation (5.5)], with the total thermal energy E_{th} replacing the total kinetic energy T (W. J. M. Rankine 1853)[2]

$$2E_{\rm th} + U = 0. \tag{5.5}$$

Thus, neglecting (as usual) extraneous factors of order unity, we can write the Kelvin–Helmholtz timescale as

$$t_{\rm KH} = \frac{Gm^2}{rL}. \tag{5.6}$$

[2] W. J. Macquorn Rankine already stated in 1850 "*that the sum of all the energies of the universe, actual and potential, is unchangeable.*" https://archive.org/details/miscellaneousscii00rank/page/202/mode/1up?view=theater.

For the Sun (`m = 1 | units.MSun`, `r = 1 | units.RSun` and `L = 1 | units.LSun`), we have

```
def Kelvin_Helmholtz_timescale(mass, radius, luminosity, G=constants.G):
    return G * mass**2 / (radius * luminosity)
```

in which case $t_{\rm KH} \simeq 31$ Myr.

5.1.1.3 Nuclear Fusion and the Nuclear Timescale

The collapse of a protostellar clump toward the main sequence proceeds on a Kelvin–Helmholtz timescale. During its slow contraction, a star follows the pre-main sequence or *Hayashi track* (Hayashi 1961). During this period, the star decreases in luminosity but stays at roughly the same surface temperature. The central temperature and pressure increase until nuclear fusion begins to contribute considerably to the total energy budget. The central density and temperature then exceed $\rho_c \gtrsim 7$ g cm^{-3} and $T_c \gtrsim 10^7$ K.

On the main sequence, where the star spends most of its lifetime, hydrogen (H) fuses into helium (He) through a variety of channels that may be summarized schematically by

$$4(^1\mathrm{H}) \rightarrow {}^4\mathrm{He} + 2\nu_e + 26.7 \text{ MeV}, \tag{5.7}$$

where ν_e represents an electron neutrino that streams out of the core, through the star, and into interplanetary space. The 26.7 MeV represents the net energy liberated as a result of this reaction. Overall, a fraction $\epsilon \simeq 0.7\%$ of the total nuclear mass is converted into energy in the form of gamma-rays. That energy ultimately contributes to the star's luminosity. Assuming that the innermost 10% of the star's mass is fusing, and assuming the above efficiency, we can write a timescale for nuclear fusion in a main-sequence star:

$$t_{\rm nuc} = 7 \times 10^{-3} \frac{0.1mc^2}{L} = 7 \times 10^{-4} \frac{mc^2}{L}. \tag{5.8}$$

In AMUSE, a simple estimate of the nuclear timescale for the Sun (`m = 1 | units.MSun` and `L = 1 | units.LSun`) gives

```
def nuclear_time_scale(mass,  luminosity, c=constants.c):
    return 7e-4 * mass * c**2 / luminosity
```

From now on, we will leave out the obvious imports of Python packages. This shows that the Sun has $t_{\rm nuc} \simeq 10$ Gyr. This timescale is the Sun's approximate nuclear-burning lifetime. More careful calculations yield 12.462 ± 0.002 Gyr (based on calculations you can do yourself in Section 5.5).

Once the hydrogen in the stellar core has fused into helium, a star starts to fuse helium into carbon (C):

$$3\,{}^4\mathrm{He} \rightarrow {}^{12}\mathrm{C} + 7.3 \text{ MeV} \tag{5.9}$$

(the reaction is somewhat more complicated than indicated here, but this is the outcome). Subsequent fusion stages form oxygen (O) and heavier elements (such as Neon, Ne) by alpha capture:

$$^4\mathrm{He} + ^{12}\mathrm{C} \rightarrow ^{16}\mathrm{O} + 7.2\ \mathrm{MeV}, \tag{5.10}$$

$$^4\mathrm{He} + ^{16}\mathrm{O} \rightarrow ^{20}\mathrm{Ne} + 4.7\ \mathrm{MeV}, \tag{5.11}$$

etc. This progression is sometimes called the *alpha ladder*. How far reactions proceed depends on the mass of the star. The Sun will never make it past oxygen, but the most massive stars will keep producing heavier elements up to iron and nickel. Fusion eventually stops at nickel (^{56}Ni), after which the reaction becomes endothermic: ^{56}Ni does not produce energy when fused.

Figure 5.4 presents the luminosity generated in the stellar interior as a function of time for three stellar masses and at two different moments in time. In each of these cases, the core is easily seen in the sudden drops in density and luminosity.

5.1.2 Physics of the Interior

Unlike the case with self-gravitating systems, we cannot easily solve the fundamental physical equations governing stellar structure and evolution. That would require us to resolve detailed nuclear reaction networks, the ways in which matter and energy flow around the star, and the details of how radiation interacts with matter.

The cross sections for interactions between radiation and matter are in fact quite well known, and nuclear reaction rates can in principle be measured in the laboratory. The reaction rates used in stellar-evolution calculations are generally extrapolated from laboratory measurements made at higher temperatures and much lower densities. The fusion reactor ITER[3] will have an operational temperature of 1.5×10^8 K and an electron number density of $n_e \sim 10^{18} - 10^{21}$ m^{-3}, corresponding to an equivalent hydrogen density of $\sim 10^{-12} - 10^{-9}$ g cm^{-3}. For comparison, the temperature and density of the solar core are roughly 1.5×10^7 K and 150 g cm^{-3}, respectively.

5.1.2.1 Underlying Assumptions in Stellar-evolution Models

Like many gravity solvers, stellar-evolution codes make a number of simplifying assumptions. Some are hidden in the mathematical or numerical models, while others are more explicit. Instead of solving for the microphysics, we generally solve for the macroscopic state of the star, which can only be realized by neglecting several aspects of the physics. The neglect of these effects does not (seem to) affect the zeroth-order calculation for most stars, and to some extent can be included as a first-order correction. Here, we list some of the most important assumptions.

1. **Spherical symmetry**. The assumption of spherical symmetry is a sweeping approximation that reduces the stellar structure problem from a complex three-dimensional set of partial differential equations to a one-dimensional set of ordinary differential equations. In the process, we lose the possibility of

[3] See https://www.iter.org

including stellar rotation, internal magnetic activity, and any other nonradial internal motion. In this approximation, the star is viewed as a series of one-dimensional arrays of macroscopic parameters, such as density, temperature, mean molecular weight, etc., stored in radial bins running from the center to the surface.

This greatly simplifies the calculation, but most actual stars are observed to have magnetic fields of varying strengths, do rotate, and exhibit a wealth of nonspherical features. One only has to look at an image of the Sun, with its many sunspots and prominences, to realize that spherical symmetry is only an approximation (see Figure 5.5). (On the other hand, solar activity—even at its maximum—is only a tiny fraction of the Sun's energy budget).

2. **Local thermodynamic equilibrium**. Radiation plays an important role in transporting energy through a star. Let us temporarily ignore the possibility of convection (see below), and assume that all energy generated in the stellar core moves through the stellar interior in the form of radiation. If the total luminosity within a sphere of radius r is $L(r)$, then the energy flux across radius r is

$$F(r) = \frac{L(r)}{4\pi r^2}. \tag{5.12}$$

In equilibrium, all the luminosity produced inside the star eventually reaches the surface and escapes into space.

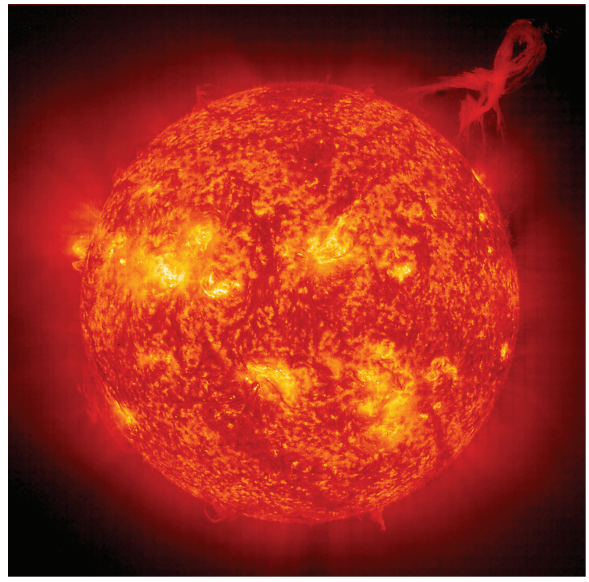

Figure 5.5. Surface image of the Sun (image from the SOHO mission at EIT 304 Å on 2000 January 18). The light and dark orange patches (as opposed to the yellow areas, which are regions of solar activity) show granulation, the result of temperature variations due to convection just below the surface. Courtesy of SOHO/[EIT] consortium. SOHO is a project of international cooperation between ESA and NASA.

The outward energy flux F is driven by a temperature gradient within the star. Basically, if the temperature decreases with radius, then at any point in the star, there will be slightly more photons moving outward from within than moving inward from outside. However, a system in thermodynamic equilibrium would have constant temperature throughout, and therefore cannot have temperature gradients. Therefore, a star cannot be in perfect thermodynamic equilibrium. Instead, we must assume that the star is very close to thermodynamic equilibrium at any radius, but there is a slight radial temperature gradient driving the outward flux. This assumption is called *local thermodynamic equilibrium*.

The mean free path of a photon is much smaller than the stellar radius, so each photon interacts frequently with local stellar material on its way out. This supports the assumption that the gas and the radiation are in local thermodynamic equilibrium: at any given point within the star, the radiation and the gas have the same temperature, even though the temperature decreases with radius. Note that these assumptions do not necessarily apply in other circumstances, such as in young neutron stars or black holes.

3. **Convection and mixing**. Radiation is not the only energy transport mechanism operating in most stars. Convection is another important mechanism that plays a vital role in determining stellar structure and evolution. Stars of very low mass tend to be fully convective, the Sun has a convective envelope, and massive stars tend to be convective in their central regions. The imprint of convection on the surface of the Sun is illustrated in Figure 5.2, and is nicely visible in Figure 5.5.

   When the energy flux through part of a star cannot be transported efficiently by radiation (e.g., when the flux becomes very large, or in a region where the opacity is very high), the energy is instead transported by convection—turbulent motion in the stellar interior. Stellar material is heated, rises, cools, and sinks back, forming convective cells that clearly cannot be properly described in a one-dimensional spherical stellar model: How can two parcels of gas pass one another on a line?

   Turbulence gives rise to many numerical complications and is not realistically resolved in current state-of-the-art one-dimensional stellar structure and evolution models. Rather, it is handled in an ad hoc fashion, assuming that convective velocities are sufficient to carry the required energy flux and that the radial extent of a convective cell is a fixed fraction of the pressure scale height, $\Delta r = \alpha P/|dP/dr|$, where α is a parameter that can be adjusted to calibrate models against the Sun. It is generally assumed that any region of a star that becomes convectively unstable is mixed by the convection process.

   Convection is particularly important in the earliest stellar-evolution phases (e.g., along the Hayashi track before the star reaches the ZAMS) and in much later phases (e.g., when the star reaches its maximum radius along the asymptotic giant branch; AGB), but some regions of most stars will become convectively unstable at some time during their evolution.

   Magnetic fields are generally considered to have a relatively small effect on the overall structure and evolution of most stars (even in white dwarfs or

neutron stars), and are therefore generally neglected. If some form of magnetism is included in the stellar-evolution code, it often follows from perturbations or through simplified models rather than full magnetohydrodynamic (MHD) calculations (see, e.g., Section 10.2.1, where we adopt a MHD model for a collapsing giant molecular cloud). Some stellar-evolution codes include such equations for the evolution of toroidal and poloidal magnetic field components derived from mean-field MHD (Takahashi & Langer 2021). These implementations often use Alfvén's theorem to formulate a conservative form of angular momentum transfer due to the Lorentz force.

Stellar material can be efficiently redistributed by convection. For example, nuclear-processed elements can be transported to the stellar surface, or fresh hydrogen from the stellar envelope can be brought into the core. The latter process is generally identified with convective overshoot (where the momentum of the convected material carries it beyond the unstable region, into the overlying stable gas) and has the effect of prolonging the lifetime of a star.

4. **Mass loss by stellar winds**. In addition to radiating energy into space, stars also lose mass. The amount of mass lost is very hard to calculate from first principles. Consequently, stellar winds are generally implemented as ad hoc prescriptions calibrated against observations (which are also uncertain). One problem here is that the stars that lose the most mass generally have the shortest lifetimes. As a result, the number of examples against which approximate methods can be calibrated is quite limited. Our imperfect theoretical understanding of the physics and the expense of a detailed numerical prescription means that mass loss in stellar-evolution codes is often parameterized, rather than solved explicitly.
5. **Equation of state**. Although it is not, strictly speaking, an assumption, any calculation of stellar structure requires an equation of state to relate the gas pressure (P) to the temperature (T) and density (ρ).

   In many cases, an ideal gas equation of state may be used. This relation is simple: at sufficiently high temperatures, the gas is fully ionized and is described well as an ideal gas, in which case (Clapeyron 1834; Clausius 1857)

$$P = nkT = \frac{\rho kT}{\mu}. \tag{5.13}$$

   Here, n is the number of particles per unit volume, k is the Boltzmann constant (not to be confused with the Stefan–Boltzmann indicated with σ in Equation (5.1)), and μ is the mean molecular weight. For a fully ionized gas with hydrogen fraction (by mass) X, helium fraction Y, and heavy element fraction $Z = 1 - X - Y$, because each hydrogen atom contributes one electron, each helium atom contributes two electrons, and (we assume) each heavy atom of atomic weight A contributes on average $A/2$ electrons, the total number density is

$$n = \frac{\rho}{m_H}\left(2X + \frac{3}{4}Y + \frac{1}{2}Z\right), \tag{5.14}$$

where $m_{\rm H}$ is the mass of a hydrogen atom. Setting $Z = 1 - X - Y$, we find

$$\mu = 2m_{\rm H}\left(1 + 3X + \frac{1}{2}Y\right)^{-1}. \tag{5.15}$$

For solar ZAMS composition, $X = 0.7$, $Z = 0.02$, and $\mu \simeq 0.62 m_{\rm H}$.

Other pressure terms may be important in some circumstances. In hot stars, radiation pressure must be taken into account, so the expression for the pressure becomes

$$P = \frac{\rho k T}{\mu} + \frac{1}{3}aT^4, \tag{5.16}$$

where $a \equiv 4\sigma/c$ is the radiation constant. This equation of state is a good description of most of the interiors of most stars for most of their lives. At very high densities, quantum mechanical electron degeneracy pressure may dominate:

$$P_e = \left(\frac{3}{8\pi}\right)^{2/3} \frac{h^2}{5m_e} n_e^{5/3}. \tag{5.17}$$

Here, n_e is the electron number density and m_e is the electron mass. Degeneracy pressure is particularly important in the cores of evolved stars and remnants. Finally, near the stellar surface, the gas may be only partly ionized and the ionization fraction of each species must be computed from the Saha–Langmuir equation (Saha 1921; Kingdon & Langmuir 1923). In addition, molecules may form, further complicating the calculation of μ.

5.1.3 Final Stages of Stellar Evolution

A star continually fights its own gravitational pull, but gravity inevitably wins, sooner or later causing the central parts of the star to turn into a compact remnant—a white dwarf, neutron star, or black hole.

In low- and intermediate-mass stars—those with masses at the end of the giant branch of less than about 8 $M_\odot$—the core grows until it reaches a state where electron degeneracy pressure prevents further contraction, and fusion ends somewhere along the alpha ladder (see Section 5.1.1.3). How far the star ascends the alpha ladder depends on the temperature and density in the stellar interior. Figure 5.6 shows the central temperature as a function of the core density. Stars are born in the lower left corner, at low temperature and density. In this regime, the star fuses hydrogen (see Section 5.7). As the star ages, its central density and temperature increase, allowing it to ascend the ladder. Only the most massive stars reach the top rung, fusing ^{52}Fe to ^{56}Ni. The portion of the script that carries out the evolutionary calculation is shown in Code example 5.1.

The core density and temperature of the most massive star in Figure 5.6 grow steadily, whereas the stars of lower mass make some interesting excursions during their evolution. The differences between these tracks are caused by the various energy transport mechanisms and changes in the equation of state in the stellar

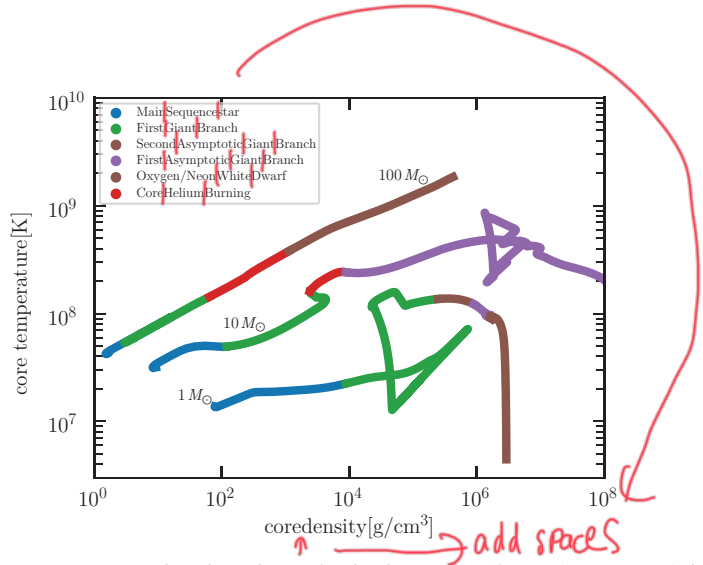

Figure 5.6. Central temperature as a function of core density for a 1 $M_\odot$ (bottom), a 10 $M_\odot$ (middle), and a 100 $M_\odot$ star (top) of solar metallicity. The colors identify various evolutionary phases (see legend), starting with the main sequence at the lower left to giants and white dwarfs to the right. The code to produce this plot can be found in `amuse.examples.stellar.calculate_core_temperature_density` (see Code example 5.1). To plot the results, it may be more practical to use the precalculated data in the simple plotting script `amuse.examples.plot_core_temperature_density`, because the Henyey stellar-evolution code `MESA` (see Table 5.1) used to generate this figure takes a few hours for each single star. In particular, the 1 $M_\odot$ star tends to be slow along the AGB just before the green part of the curve; magnifying this part of the evolution will reveal interesting sinusoidal variations in the core density and core temperature of the star.

```
    while not stellar_remnant(stellar):

        star.evolve_one_step()

        nzones = star.get_number_of_zones()
        rhoc = star.get_density_profile(nzones)[0]
        Tc = star.get_temperature_profile(nzones)[0]

        time.append(stellar.model_time)
        rho_core.append(rhoc)
        T_core.append(Tc)
        stellar_type.append(star.stellar_type)
        si = star.stellar_type.value_in(units.stellar_type)

        with open(filename, "wb") as file:
            pickle.dump([time, rho_core, T_core, stellar_type], file)

```

Code example 5.1: Calculating the core temperature and density of a stellar model (used to make Figure 5.6). The full script can be found in `amuse.examples.calculate_core_temperature_density`.

interior. For the Sun and other low-mass stars, the alpha ladder stops when carbon and helium fuse to form oxygen; for more massive stars, it ends at oxygen and/or neon (see Equation (5.10)). Eventually, nuclear fusion stops because (due to electron degeneracy) the contraction of the stellar core cannot produce the density and

temperature required to reach the next rung of the alpha ladder. As the star runs out of fuel, instabilities arise and propagate to the outer layers, eventually ejecting them into space in the form of a dense stellar wind. This final phase is called the post-AGB. The leftover cinder of processed material becomes a white dwarf, the composition of which depends on how far the star ascended the alpha ladder.

The evolution of a higher-mass star ends much more violently. Once all available fuel is spent, and the stellar core is mainly composed of iron-group (Scandium ^{45}Sc to Nickel ^{56}Ni) elements, further fusion is not possible. As the core mass grows due to nuclear fusion in the overlying layers, the only remaining route is collapse. The resulting phenomenon is called a supernova, although this term is also often applied to the core-collapse process. A wide variety of supernovae have been identified observationally. Broadly speaking, we can recognize two basic types, II and Ib/c, but many subcategories have been defined (Smartt 2009). To make a rough distinction, type II supernovae are the result of core collapse in a massive hydrogen-rich star, as just described, leading to the formation of a neutron star or black hole (Chiu 1964). Type Ib/c supernovae result from the collapse of helium-rich (Wolf–Rayet) or stripped stars and may lead to black hole formation, although this remains uncertain (Georgy et al. 2009; Langer 2012; Laplace et al. 2020). There is a hypothetical class of homogeneously evolving stars (Laplace et al. 2020); though never observed, they are an interesting potential aberration of stellar-evolution calculations. Type Ia supernovae may mark the collapse of a massive white dwarf, possibly formed via a white-dwarf merger, or through accretion from a companion star.

Supernovae are not necessarily symmetrical, and the compact object may receive a velocity kick as a result of the explosion. This is often referred to as a *natal kick* of the neutron star or black hole in a type Ib/c or type II supernova. These velocity kicks are uncertain, but may be as high as 1000 km s^{-1} (Helfand & Tademaru 1977; Lyne & Lorimer 1994; Kapil et al. 2023).

5.2 Simulating Stellar Evolution

The most detailed stellar-evolution codes solve the underlying (one-dimensional) differential equations for stellar structure and evolution (for details, see Eggleton 2006). They advance the nuclear burning and stellar structure equations at every time step, and by taking energy transport and nuclear energy generation into account, they converge to a time-dependent solution. These methods are accurate, within the simplifying assumptions listed earlier, but relatively slow. As a result, a second class of much more rapid methods has appeared. They use fits to the more detailed results, generally expressed in the form of look-up tables (Meynet et al. 1994; Meynet & Maeder 2005; Brott et al. 2011; Köhler et al. 2015), fitting formulae (Eggleton et al. 1989; Hurley et al. 2000), or other parameterizations (Varshavskii & Tutukov 1975; Tutukov & Fedorova 2001; Spera et al. 2015). We refer to the first class of stellar-evolution codes as direct solvers. The second class, we call parametric stellar-evolution prescriptions; both are available in AMUSE.

Direct stellar-evolution solvers use so-called Henyey stellar-evolution models (Henyey et al. 1959, 1964) providing a one-dimensional representation of a star

using radial distributions of stellar mass, density, composition, etc. Hybrid Henyey codes exist, in which the basic differential equations are expanded to include atmosphere models, rotation, and some form of convective overshoot and mixing, but those models are, strictly speaking, still one-dimensional (Schaerer et al. 1996; Stasińska & Schaerer 1997). Research on true three-dimensional stellar-evolution code is still very limited (Bazán et al. 2003; Rizzuti et al. 2023).

Parametric stellar-evolution methods come in two varieties: those adopting functional forms of fits to earlier calculated stellar-evolution tracks, and look-up tables. Fitting formulae adopt polynomials with mass and sometimes composition—typically expressed as metallicity—as free parameters (see Section 5.2.3), and yield the stellar radius, temperature, and luminosity as functions of time since the ZAMS. Look-up tables tend to interpolate between tabulated values for stellar mass, radius, luminosity, etc., as a function of time and metallicity. Parametric stellar-evolution models generally have very limited internal structure information. Some include some notion of a core–envelope structure, and possibly additional parameters such as the radius of gyration, but their structural content is minimal.

Look-up parametric stellar-evolution methods (indicated in Table 5.1 as *parametric*) generally interpolate from precomputed tables of stellar-evolution tracks (mass, radius, temperature, luminosity, etc.). In some sense, the look-up and parametric stellar-evolution codes are equivalent, but they differ in their computational requirements in terms of both memory management and CPU performance. In general, however, both are much faster than Henyey codes. The evolution of a 1 $M_\odot$ star from ZAMS until it leaves the AGB takes about a second for a parametric code and up to a day for a Henyey code.

5.2.1 Initial Conditions for Stars

A star is born—hydrogen fusion starts—with a certain mass and metallicity, at which point we call it a ZAMS star. For a parameterized stellar-evolution code (see Section 5.2), the specification stops there—the future evolution of the star is completely determined via fitting formulae and/or tables.

For a Henyey code (see Section 5.2) these parameters generally also describe the star, but a number of additional parameters must also be specified. In this type of code, the nuclear reaction network, the hydrodynamics, and the radiative transport are generally solved in a number of shells—a star in a Henyey code is layered like an onion (Steig 2008). One of the parameters in such a code is the number of layers (onion shells) to use in the code's description of stellar structure. In general, we must specify the run of temperature, density, etc. throughout the (one-dimensional, spherically symmetric) star. The `MESA`, `EVTwin`, and `GENEC` stellar-evolution codes can generate initial stellar models based on mass and metallicity, but we can also have these codes start from completely arbitrary structures, under our control. It is possible, for example, to start with a pre-main-sequence star, in which case the evolution starts somewhere on the Hayashi track (see Chapter 5), or with a giant planet. Another interesting initial stellar model is the hydrogen-exhausted core of a giant without an envelope, which could be interpreted as a zero-age helium

Table 5.1. Stellar-evolution Codes Incorporated in AMUSE

Name	Ref	Type of Code	Language	Rotation Evolution	Binary Structure	Internal	Parallel
MESA	1	Henyey	F95	Yes	Yes	Yes	Yes
EVTwin	2	Henyey	F77	No	Yes	Yes	No
GENEC	3	Henyey	F90	Yes	No	No	No
SSE/Bse	4	Parametric	F77	No	Yes	No	No
SeBa	5	Parametric	C++	No	Yes	No	Yes
Toy	6	Scaling relations	C++	No	No	No	No

1: Paxton et al. (2010, 2011); 2: Eggleton et al. (2011); Eggleton (1971, 2006); 3: Maeder & Meynet (1988); Ekström et al. (2012); Groh et al. (2019), https://martseba.github.io/; 4: Hurley et al. (2000); 5: Portegies Zwart & Verbunt (1996); Portegies Zwart & Yungelson (1998); Toonen et al. (2012); Portegies Zwart & Verbunt (2012); 6: (Hut et al. (2003)).

main-sequence star (these days often called a stripped star because they naturally follow from an episode of type B mass transfer; Kippenhahn & Weigert 1967). Yet another interesting study are chemically homogeneous stars, which are sometimes considered as progenitors for black hole mergers (Brott et al. 2011).

Code example 5.2 presents a simple script to initialize a star of mass M and metallicity z, and plots the associated density profile. Figure 5.7 shows the density structure of an $M = 2\ \mathrm{M}_\odot$ star simulated using two of the Henyey codes, `EVTwin` and `MESA`. In addition, we plot the structure of two $1\ \mathrm{M}_\odot$ stars that merge into a single star at a very early stage in their evolution (see Chapter 5 and Code example 5.5 for details). Such a collision would provide a good reason to start with a nonstandard stellar initial model. In this case, one could use `MakeMeAMassiveStar` (MMAMS; Lombardi et al. 2002; Gaburov et al. 2008) to generate a stellar initial model.

```
def get_density_profile(code=Evtwin, mass=2.0 | units.MSun, metallicity=0.02):
    stellar = code()
    stellar.parameters.metallicity = metallicity
    stellar.particles.add_particle(Particle(mass=mass))
    radius = stellar.particles[0].get_radius_profile()
    rho = stellar.particles[0].get_density_profile()
    stellar.stop()
    return radius, rho
```

Code example 5.2: Script to initialize a single ZAMS star using `EVTwin`. Code fragment from `amuse.examples.initialize_single_star`.

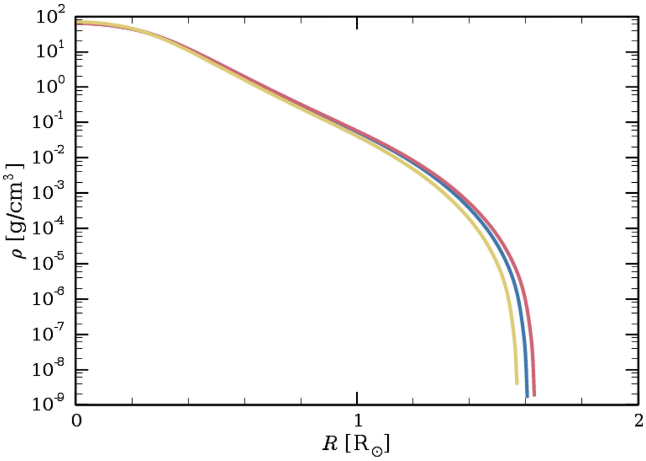

Figure 5.7. Density profile of a star on the ZAMS. All three curves are for a single $2\ \mathrm{M}_\odot$ star. Two curves are calculated using the stellar structure and evolution codes `EVTwin` (blue curve) and `MESA` (red). The yellow curve is generated from a merger of two $1\ \mathrm{M}_\odot$ stars, which are combined conservatively using `MMAMS` for constructing a $2\ \mathrm{M}_\odot$ in dynamical equilibrium, whose structure is then calculated using `MESA`. The merger calculation is performed using the script in Code example 5.5. The script to generate this figure can be found in `amuse.examples.stellar.merger_stellar_density_profile`.

```
def merge_two_stars(
    mass_primary=5 | units.MSun,
    mass_secondary=3 | units.MSun,
    time_collision=1.0 | units.Myr,
):
    """
    Merges two stars using the Make Me a Massive Star code and returns the
    resulting density profile.
    """
    primary = Particle(mass=mass_primary)
    secondary = Particle(mass=mass_secondary)

    stellar = Mesa()
    primary = stellar.particles.add_particle(primary)
    secondary = stellar.particles.add_particle(secondary)

    stellar.evolve_model(time_collision)

    stellar.merge_colliding(
        primary.copy(), secondary.copy(), Mmams, return_merge_products=["se"]
    )
    radius = stellar.particles[0].get_radius_profile()
    rho = stellar.particles[0].get_density_profile()
    stellar.stop()
    plot_density_profile(radius, rho)
```

Code example 5.5: Simple routine to merge two stars (see `amuse.examples.merge_two_stars`). The script should run in about a minute on a laptop.

It is interesting to note that the profiles calculated by the three codes are not the same. The largest differences are in the outer parts of the stars, and probably these do not make a huge difference to global stellar evolution. But they may affect quite dramatically how the star appears in an H–R diagram. A larger star translates to a dimmer and cooler stellar surface.

5.2.2 Stellar-evolution Modules in AMUSE

Currently, in AMUSE we can choose between three Henyey codes, two parametric prescriptions for stellar evolution, and three binary evolution codes. Table 5.1 presents some fundamental parameters for these codes.

Just as we can easily run a simple N-body simulation in AMUSE, we can also run a simple stellar-evolution calculation. The snippet in Code example 5.3 looks quite similar to the code presented in Code example 4.1 of Section 4.1.1, and indeed, the interface function names are deliberately chosen to be similar from one module to another. Code example 5.3 omits the details of parsing the command line to obtain the desired parameters and passing them to the `main` function. To incorporate that, we could include the Python argument parser, as discussed in Appendix A (see Section 2.4.2).

Instead of declaring a gravitational N-body code, as we did in Section 4.1.1, here we initialize the stellar-evolution code, and add a single particle of mass `m`, as follows:

```
from amuse.units import units
from amuse.datamodel import Particle
from amuse.community.seba import Seba


def stellar_minimal(
    mass=1.0 | units.MSun, metallicity=0.02, time_end=4700 | units.Myr
):
    stellar = Seba()
    stellar.parameters.metallicity = metallicity
    star = stellar.particles.add_particle(Particle(mass=mass))

    initial_luminosity = star.luminosity
    time_step = 0.1 | units.Myr

    while stellar.model_time < time_end:
        stellar.evolve_model(stellar.model_time + time_step)

        print(
            f"at T={stellar.model_time.in_(units.Myr)} "
            f"L(t=0)={initial_luminosity}, "
            f"L (t={star.age.in_(units.Myr)})={star.luminosity.in_(units.LSun)}, "
            f"m={star.mass.in_(units.MSun)}, "
            f"R={star.radius.in_(units.RSun)}"
        )

    luminosity = star.luminosity

    print(
        f"Time={stellar.model_time.in_(units.Myr)} "
        f"L={np.log10(luminosity.value_in(units.erg / units.s))}"
    )

    stellar.stop()
```

Code example 5.3: Minimal routine for evolving a single star of mass M and metallicity z to some end time `model_time`. This snippet was taken from the source code in `amuse.examples.stellar_minimal`.

```
stellar = Seba()
stellar.particles.add_particle(Particle(mass=mass))
```

The default metallicity for most codes is the solar value: $z = 0.02$, but as can be seen, this parameter is easily changed, which we explain in Section 5.2.3.

The stellar-evolution code chosen here is `SeBa`, which is a parametric stellar-evolution code (see Table 5.1). The single star is added to the repository of stars in the running `SeBa` instantiation. In the line

```
initial_luminosity = stellar.particles.luminosity
```

we fetch the stellar luminosity directly from the stellar-evolution code. We do something similar in the eventual print statement

```
print(
    f"at T={stellar.model_time.in_(units.Myr)} "
    f"L(t=0)={initial_luminosity}, "
    f"L (t={star.age.in_(units.Myr)})={star.luminosity.in_(units.LSun)}, "
    f"m={star.mass.in_(units.MSun)}, "
    f"R={star.radius.in_(units.RSun)}"
)
```

where we print some of the stellar properties after the star has been evolved to `model_time`. This is mainly a matter of convenience. For efficiency, we might alternatively have used a `copy()` function on a predefined communication channel, as discussed in Section 2.2.3.

Stars in AMUSE are categorized into types, from pre-main sequence to black hole. This parameter is called the `stellar_type`. It is convenient for referring to the evolutionary state, but it should only be used as a rough measure of the stellar structure. We provide a complete list of AMUSE stellar types in Appendix A.

5.2.3 Changing the Metallicity of a Star

Stellar-evolution codes are tuned to the Sun. Many stars, probably most, have a different metallicity. For historical reasons, metallicity is a code parameter, rather than a characteristic of the star. We can change the metallicity of the stellar-evolution code by setting the appropriate parameter. As a consequence, all stars in that particular code will be evolved according to that specific metallicity. In order to run multiple stars with different metallicities, we would have to instantiate multiple stellar-evolution codes, as we will demonstrate later in Section 5.2.6.

Changing the stellar metallicity in any of the stellar-evolution codes is easy. For example, we can start `SeBa` with one tenth solar metallicity ($0.1z_\odot$) by

```
stellar = Seba()
stellar.parameters.metallicity = 0.002
```

or equivalently, `EVTwin`

```
stellar = Evtwin()
stellar.parameters.metallicity = 0.002
```

Both codes can be provided with a star and evolved

```
stellar.particles.add_particles(Particles(mass=1 | units.MSun))
stellar.evolve_model(1 | units.Myr)
```

which for `EVTwin` leads to a radius of $2.93\,\mathrm{R}_\odot$.

For a different metallicity, it is often better to start the stellar-evolution code from a pre-main-sequence star because prebuilt models are only stored in the AMUSE repository for solar metallicity.

5.2.4 Starting a Star on the Pre-main-sequence

Henyey stellar-evolution codes can as easily start on the pre-main-sequence, or at any stellar stage, so long as there is a consistent input stellar-evolution model. The Henyey codes will construct such a model if it is not readily available, although this may consider a considerable amount of computing time.

We can, for example for example initiate the `MESA` stellar-evolution code at solar metallicity.

```
stellar = Mesa(version="2208")
```

generates an instantiation of `MESA`. Here, we adopted `MESA` version 2208, as we explicitly indicated as an argument. Now, give it a $1\,M_\odot$ star to work with and evolve it for 1 Myr:

```
stellar.pre_ms__stars.add_particles(Particles(mass=1 | units.MSun))
stellar.evolve_model(1 | units.Myr)
```

Or give it a $1\,M_\odot$ pre-main-sequence star to work with, and evolve it for 1 Myr:

```
stellar.native_stars.add_particles(Particles(mass=1 | units.MSun))
stellar.evolve_model(1 | units.Myr)
```

At an age of 1 Myr, after the ZAMS, we can obtain the radius of the $1\,M_\odot$ star with

```
print(f"Stellar radius={stellar.particles.radius}")
```

returns

```
Stellar radius=[0.893773591284] RSun
```

for the star that was evolved for 1 Myr after the ZAMS, and

```
Stellar radius=[9.03388865301] RSun
```

for the pre-main-sequence star at an age of 1 Myr after the onset of Hydrogen burning. The pre-main-sequence star is, as expected, considerably larger than the star that started on the ZAMS. Even after 50 Myr, the pre-main-sequence star is still somewhat larger than the ZAMS stellar radius. But the pre-main-sequence star, by that time, has lost a little mass (about four Earth masses).

We can do the same for `EVTwin`, which gives

```
Stellar radius=[0.895857558887] RSun
```

for the `1 | units.MSun` star of solar metallicity. Starting on the pre-main-sequence in `EVTwin` regrettably does not work in the same way as in `MESA`, but requires a separate function call:

```
stellar = Evtwin()
stellar.new_particle_method(pms=True)
stellar.particles.add_particles(Particles(mass=1 | units.MSun))
stellar.evolve_model(1 | units.Myr)
```

5.2.5 Improving the Stellar-evolution Solver

We can improve the stellar-evolution solver by adding two new features to the code. The first is to store the results for later use. This is accomplished in much the same way as when we evolved a cluster of stars in the gravity solver and graphed the results (similar to the snippet presented in Code example 4.3). After storing the stellar data at the end of each step, we can plot it immediately in the same script, or save it and use a second small Python/AMUSE script to visualize the results.

Evolving a single star is somewhat limited. The second improvement is to evolve an entire population to study the isochrones of the stellar system—shown as curves on the H–R diagram showing stars of the same age. We can generate a list of masses from a stellar mass function (see Section 3.3, Equation (3.1)). Any distribution will do, but from an observational perspective, the Salpeter (1955) mass function, a power law with index `alpha = -2.35`, is popular (see Section 3.3). AMUSE contains a special function, `new_salpeter_mass_distribution`, to initialize masses according to this distribution, but here we use a more general power-law mass function to accomplish the task. The code

```
m = new_powerlaw_mass_distribution(N, Mmin, Mmax, alpha=-2.35)
stellar.particles.add_particles(Particles(mass=m))
```

returns an array of `N` masses chosen randomly between `Mmin` and `Mmax` with a Salpeter power-law distribution, and it then adds them to the stellar-evolution solver. For a more general discussion on stellar mass functions, we refer the reader to Section 3.3.

Figure 5.8 is an H–R diagram of 1000 stars drawn from a Salpeter mass function with initial masses between 0.1 $M_\odot$ and 100 $M_\odot$, evolved to an age of 4.5 Gyr. Due to the random sampling, the minimum mass turned out to be 0.100008472297 $M_\odot$ and the maximum is 7.85091223641 $M_\odot$. We assumed solar metallicity and evolved the stars using `SeBa` and `MESA`. The Henyey code crashed on several of the stars, particularly those of mass $\gtrsim$1.2 $M_\odot$; we have omitted these stars from Figure 5.8. In Section 8.3.3, we discuss how AMUSE deals with a crash of one of the underlying codes. Table 5.2 presents a small performance comparison between the four AMUSE stellar-evolution modules.

5.2.6 Evolving an Inhomogeneous Stellar Population

In the stellar-evolution prescriptions implemented in AMUSE, time and metallicity are properties of the code, rather than of the star. This is somewhat confusing, but it is how the Henyey codes are structured. Changing the metallicity, in particular, requires the code to load a new set of opacity tables describing the stellar interior, and this is simply too time-consuming to do efficiently every time we switch our

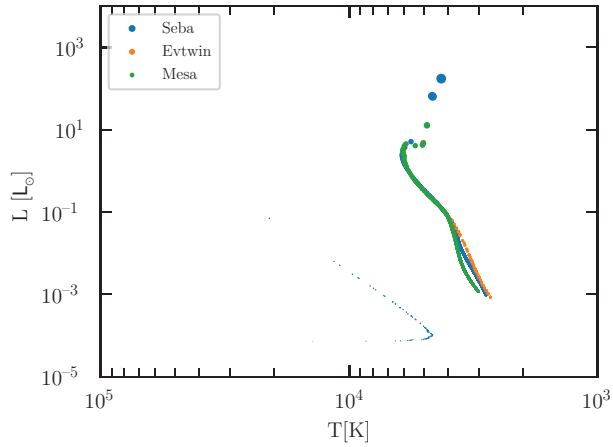

Figure 5.8. H–R diagram of 3000 stars with solar metallicity and initial masses between 0.1 $M_\odot$ and 100 $M_\odot$, evolved to an age of 4.5 Gyr. The stars were evolved using `SeBa` (blue), `MESA` (green), and `EVTwin` (red); the symbol sizes are proportional to the square root of the stellar radius. Only `SeBa` managed to evolve the stars through the white-dwarf phase. This figure was produced with script `amuse.examples.stellar_isochrone`. Using `MESA` and `EVTwin`, the evolution of stars to the remnant stage is time-consuming; this script will take many days to run, even on a multicore workstation.

Table 5.2. Comparison of the Four Available Stellar-evolution Codes in AMUSE

Name	$t_{\rm CPU}$	$M/{\rm M}_\odot$	${\rm R}/{\rm R}_\odot$	$L/{\rm L}_\odot$	T/K
Mesa	0:35.29	0.996	4.11	8.01	4792
EVTWIN	3:20.07	0.993	18.6	98.0	4212
SSE/BSE	0:0.63	0.999	2.58	3.40	4884
Seba	0:0.62	0.999	2.58	6.40	4878

Notes: Here, we calculated the evolution of a 1 $M_\odot$ solar-metallicity star to an age of 11.75 Gyr. For `EVTwin`, the 1 $M_\odot$ star turned into a white dwarf slightly after this, and therefore takes considerably more CPU time than the other Henyey codes. The runs were performed on a 2.4 GHz Core-i7 processor. CPU times are measured in hours:minutes.

attention from one star to another. An analogy might be the time in an *N*-body code, which also is an attribute of the code rather than of the individual particles. Thus, evolving a stellar population of a single age or metallicity is easy, but a multiage or multimetallicity population requires a little extra work.

The easiest way to evolve a population of stars of different ages is by instantiating as many stellar-evolution codes as there are stellar age groups, or maybe even one code for each individual star. Code example 5.4 presents a small script for evolving several stars, each to a different age and with its own stellar-evolution code. When simulating a limited number of stars, the method in Code example 5.4 is probably adequate because it is simple and transparent. Performance is not really an issue, because each stellar-evolution code can run independently, which makes for an embarrassingly parallel problem. For large numbers of stars, however, the memory requirements of this approach may become prohibitive.

```
def evolve_with_different_stellar_model():
    times = [10, 100, 1000] | units.Myr
    stars = Particles(mass=[1, 2, 4] | units.MSun)
    stellars = [Seba(), Mesa(), Sse()]
    channels = []
    for star, stellar in zip(stars, stellars):
        stellar.particles.add_particle(star)
        channels.append(stellar.particles.new_channel_to(stars))

    for time, channel, stellar in zip(times, channels, stellars):
        stellar.evolve_model(time)
        channel.copy()

    for time, star in zip(times, stars):
        print(f"Time= {time} M= {star.mass}")

    for stellar in stellars:
        stellar.stop()
```

Code example 5.4: Example code to initiate three stellar-evolution codes to evolve three stars, each to a different age. This code fragment is taken from the source code in `amuse.examples.multiple_stellar_code`.

5.2.7 Enforcing Stellar Mass Loss/Gain

The stellar-evolution codes in AMUSE have predetermined mass-loss prescriptions. They are generally not very elaborate because stellar mass loss is not understood particularly well. For this reason, and in order to allow the user to enforce mass loss or gain from other sources (imposed by hand in your script), we have enabled user-specified mass changes. If you invoke this feature, it is important to realize that you will have to switch off the standard mass-loss procedures. Built-in winds on the RGB and on the AGB must be turned off via:

```
stellar.parameters.RGB_wind_scheme=0
stellar.parameters.AGB_wind_scheme=0
```

This will disable the mass-loss prescription in the instantiation of the stellar-evolution code, and all stars evolved with this code are consequently affected.

Stellar mass loss can subsequently be imposed on a particular particle `i` in the code by

```
stellar.particles[i].mass_change = dmdt
```

where `dmdt` is the mass-loss rate of star `i`. The other stars are unaffected and will not lose any mass. For example, for an Eta-Carinae-like luminous blue variable star, one might say

```
dmdt = -2.e-2 | units.MSun/units.yr
```

to model the remarkably high mass-loss rate of that star. A positive value of `dmdt` indicates that the star will gain mass; a negative value means mass loss.

Henyey stellar-evolution codes are generally not prepared for large changes in the stellar mass, and may respond unexpectedly when the user makes an adjustment. One of the major issues here is that the internal time step is calculated on the assumption that mass loss is handled internally. Incorporating a much larger mass-loss rate—for example, when simulating Roche-lobe overflow and calculating the mass loss from the radius of the Roche lobe relative to the stellar radius—may cause the internal time step to become too large to accommodate the externally imposed rate. It may even cause the stellar-evolution code to crash—which might be the best possible outcome given that the code's response to an excessive mass change may shift the evolution onto a new and entirely different track. This evolutionary track is not necessarily wrong, it could represent a viable solution to the underlying differential equations, but one may wonder if it really represents an observable star (stars are not onions; after Steig 2008).

Changing the mass-loss rate of a particular star therefore also requires changing the stellar-evolution time step of that star. This might be realized for star `i` by allowing it to lose less than `dm = -0.02 | units.MSun` within one time step, as follows:

```
new_time_step = min(dm/dmdt, stellar.particles[i].time_step)
stellar.particles[i].time_step = new_time_step
stellar.evolve_model()
```

Here, we start by calculating the new time step, after which we overwrite the star's time step with a smaller one and evolve the stars. The stellar-evolution code will try to evolve the star to the specified next time step. However, this may fail because it also has the requirement that the star remain in thermal and dynamical equilibrium, in which case the actual time step taken by the code may be different.

In our example of evolving Eta Carinae as a 90 $M_\odot$ ZAMS star, we obtain an initial evolution time step of 1.3 yr with `MESA`. When changing the mass-loss rate as just described and reducing the time step to 0.5 yr, checking the progress of the code indicates that the star was actually evolved for only 0.495 yr. After the first step is taken with the enhanced mass-loss rate and the reduced time step, subsequent time steps will be determined internally again. To guarantee that the star evolves according to the desired mass-loss rate and with subsequent small time steps, these parameters will have to be set again before the next step is taken.

In general, changing the time step will affect the calculation results, but often not by much. For example, we might imagine evolving a solar-metallicity 90 $M_\odot$ star for 1.3 yr in a single step or in multiple steps. The radius of the star when evolved in a single step is 13.423 $R_\odot$, versus 13.434 $R_\odot$ when evolved for the same amount of time, but in 10 equal-sized steps. Such a difference is seemingly unimportant, although sometimes a difference in size of just 1 $R_\oplus$ might make a difference.

5.2.8 Accessing Stellar Interiors

The direct (Henyey) stellar-evolution codes in AMUSE solve for the entire structure of the stellar interior. The internal structure of a star is stored in zones, each of which

represents a grid point at which the stellar structure equations are solved. Modern stellar-evolution codes typically use between 200 and 2000 zones. The number of zones for star `i` in the stellar-evolution code `stellar` can be queried by

```
n_zones = stellar.particles[i].get_number_of_zones()
```

Stellar interiors are accessed using the particles attribute function `get_xxx_profile()`, which returns an array of data corresponding to the stellar internal structure parameter `xxx`, which may be `mass`, `radius`, `temperature`, `luminosity`, `density`, `entropy`, `thermal_energy`, etc.

For example, the internal temperature structure can be obtained by

```
temperature_profile = stellar.particles[i].get_temperature_profile(n_zones)
```

The returned array (here called `temperature_profile`) contains the temperature profile of the star, starting at the center (the first element of the array) and running all the way to the photosphere (the last element). To see the available options, it is probably best to use the `inspect` class functionality in Python. For example, for `EVTwin`:

```
import inspect
from amuse.community.evtwin import Evtwin
print(inspect.getsource(Evtwin))
```

Note that the zones in the AMUSE stellar-evolution codes are not distributed linearly in mass, radius, or any other parameter, but are spaced to ensure optimal functioning of the code. It is therefore important to always obtain the mass or radius profile along with the desired parameter. The code snippet in Code example 5.1 shows how to access the internal density and temperature profile of a star. Other internal stellar parameters can be accessed in similar ways. The number of access functions for the stellar interior is quite large, mainly because stellar evolution is a complex process, involving a wide variety of physics.

We encourage the reader to experiment with querying the internal stellar structure, and even changing it. To reduce the temperature in the outermost shell of a 10 $M_\odot$ star, one could say

```
T_profile[-1] = 0.9 * T_profile[-1]
stellar.particles[i].set_temperature_profile(T_profile)
stellar.particles[i].time_step = 0.5 * stellar.particles[i].time_step
stellar.evolve_model()
```

Here, we also reduce the time step (arbitrarily cutting it in half), in an attempt to allow the stellar-evolution code to adjust to the change. We emphasize that it is very easy to crash a stellar-evolution code by fiddling with the stellar structure because the changes can easily cause the code to fail to converge. A typical example of output you should expect, after 117 lines of debugging messages, will eventually look like:

```
CodeException: Exception when calling function "evol-
ve_for", of code
"MesaInterface", exception was "lost connection to code"
```

Seeing how a code crashes and understanding why (here because it was forced on the code by small changes to the stellar interior) is an excellent way to appreciate the complexity of stellar-evolution calculations.

5.2.9 Modeling Stellar Mergers

Stellar structure is critical when we merge two stars. A fast and convenient approximate approach to determining the interior structure of a merger product is provided by the `MakeMeAStar` code (MMAS; Lombardi et al. 2003) and its extension MMAMS (Gaburov et al. 2008). These codes use data on the interiors of the two colliding stars to construct a new self-consistent stellar model, sorting mass shells by buoyancy to create the merger product, which can then be recommitted into a Henyey stellar-evolution code to continue the evolution. Mass loss in the merger event is calibrated against a set of independent hydrodynamical simulations (mainly using smoothed particle hydrodynamics; see Section 6.2.1). The result is a fast method for obtaining a first-order estimate of the structure of the product of a low-velocity, roughly head-on merger of two stars.

Code example 5.5 presents a simple script in which two main-sequence stars of masses `Mprim` and `Msec` are merged using `MMAMS`. Here, the primary and secondary masses are sent to the merger routine, which creates and evolves the two stars to time `t_coll` and then uses `MakeMeAMassiveStar` to merge them. The two lines

```
primary = stellar.particles.add_particle(bodies[0].as_set())
secondary = stellar.particles.add_particle(bodies[1].as_set())
```

initialize the primary and secondary stars. The particles in `bodies` contain only mass; they have no position or velocity information because we never initialize those quantities.

Note that functions such as `new_plummer_model()`, which generate initial conditions, generally return a particle set, but `Particles(...)` only returns a list. Thus, before adding the particles to the `stellar` instantiation, they first have to be converted into a particle set. In addition, the return value of `add_particles` is a pointer to the particle last added in the community code. (This was also true in the numerous previous examples using `add_particles`, but we ignored this fact in those cases.) In this case, it is convenient to maintain explicit access to the primary and secondary stars in the community code, rather than referring to them as `stellar.particles[0]` and `stellar.particles[1]`.

After the initialization of the two stars, we evolve them to time `t_coll`. Because both the `primary` and `secondary` point directly to community code memory (which happens unseen here, via the MPI channel or a socket), they contain the same internal properties of the star as in the module memory. After the evolution step, we merge the

two stars using the routine `merge_colliding` in `MakeMeAMassiveStar`. We work here with copies of the stars because both will be removed from the `stellar` particle set when the merger product is created. On return, the merger product is in `stellar_particles[0]`. The phrase `return_merge_products=["se"]` ensures that it has a Henyey structure describing its internal properties. The function `merge_two_stars` then returns the `radius` and `density` attributes of this structure. This script was used to generate Figure 5.7. In Section 5.4.2, we expand on this method by performing simulations to try to reconstruct an observed blue straggler.

This may look like a bit of dark AMUSE magic, but keep in mind that all it does is hand two stars to the merger routine and return two arrays describing the internal structure of the merger product.

5.2.10 Interrupting Stellar Evolution

As we discussed in relation to gravitational dynamics in Section 4.5.1, certain events in stellar-evolution calculations may surpass a particular code's domain of reliability. An example is the supernova explosion of a massive star, which can happen quite suddenly. The Henyey codes are not designed to continue after the supernova. We can flag and handle these events using stopping conditions, much as we discussed for gravitational dynamics in Section 4.5.1 (a more general discussion is presented in Section A.5.2). The evolution of the remnant can then be continued using another code that is better suited to the task.

We enable the stopping condition in the stellar code by

```
detect_supernova = stellar.stopping_conditions.supernova_detection
detect_supernova.enable()
```

In the event loop, we can use the following snippet:

```
if detect_supernova.is_set():
for ci in range(len(detect_supernova.particles(0))):
    print(detect_supernova.particles(0))
    code_body = Particles(particles=detect_supernova.particles(0))
    local_body = particles_in_supernova.get_intersecting_subset_in(bodies)
```

Here, `code_body` is a pointer to the exploding star in the stellar-evolution code `stellar`, while `local_body` is the same star in the local particle set. The former has the most up-to-date data from the stellar-evolution code, but the latter may contain additional information used by other modules or additional local calculations.

5.3 Binary Evolution

Stars often appear in pairs or higher multiplicities. Many of the codes in AMUSE suitable for handling single stars can also handle binaries. In all cases, however, binary evolution is something of a fix to what a single star does because there are

quite a number of assumptions underlying the evolution of binary stars. That said, binary evolution is relatively easy compared to triple evolution. In Appendix B.2.1, we briefly discuss triple evolution, as implemented in TRES, but here we limit ourselves to the fundamental principles of binary evolution.

Instead of following binary evolution through one of the available codes, one could write an AMUSE script to evolve a binary star using any of the available stellar-evolution codes. In that way, one can easily test to what degree differences in the various stellar-evolution codes lead to different binary evolution outcomes. Here is a brief recipe for such a binary evolution prescription.

Two stars evolve basically independently of each other so long as their mutual distance is considerably larger than the sum of their radii. Once the distance at pericenter [$q = a(1 - e)$] becomes smaller than about 5 times the radius of the larger star, tidal effects start to become important. Note that binaries with orbital periods $P_{\rm orb} \gtrsim 20$ year probably never reach this stage, except if they have very high eccentricities. For two $1\,{\rm M}_\odot$ stars, a 20 year period relates to an orbital distance of about 10 au. Those wide binaries are only subject to stellar wind mass loss (cf. Section 5.3.1) and possibly the effect of a supernova (cf. Section 5.3.6).

5.3.1 Stellar Winds

On the main sequence, most stars only lose a little mass, but in massive stars or stars in their late evolutionary stages, mass loss can be quite copious. Such mass loss drives the expansion of the orbit by conservation of angular momentum and energy.

If the total amount of mass lost by the most massive star is $\eta = (M - M_0)/M_0$ (Livio & Warner 1984), we write the change in orbital separation from a_0 to a as (Portegies Zwart & Verbunt 1996)

$$\frac{a}{a_0} = \left(\left[\frac{M}{M_0} \right]^{\eta} \frac{m}{m_0} \right)^{-2} \left(\frac{M_0 + m_0}{M + m} \right)^{2n_j - 1} . \tag{5.18}$$

Note that we allow the secondary star to accrete a small fraction ($m - m_0 < M_0 - M$) of the mass lost by the primary. Here, we have introduced the parameter n_j, which represents the amount of angular momentum that is carried with the material that is lost from the binary system. For an isotropic wind $n_j = 0$, but, for example, in the case of nonconservative mass transfer from a donor star to an accretor $n_j > 0$, due to the amount of angular momentum that is carried with the lost mass (see Section 5.3.4).

5.3.2 Tidal Circularization and Synchronization

Before mass transfer starts (if it starts), tidal effects tend to circularize the orbit, and synchronize the eventual donor star. Both tidal circularization and synchronization are effective, and full synchronization is generally achieved before one of the stars initiates mass transfer. Tidal circularization proceeds on a timescale $t_{circ} \sim 18,000$ yr (Duquennoy et al. 1992), during which the eccentricity $e$damps to zero (Zahn 1977)

$$\frac{de}{dt} = \frac{e}{t_{circ}}\left(\frac{P_{\rm orp}}{[1{\rm day}]}\right)^{-16/3}. \tag{5.19}$$

As the orbit circularizes, the binary's angular momentum is conserved and the orbit widens, with

$$(1 - e^2)a = \text{constant}. \tag{5.20}$$

The circularization timescale is short compared to most other processes that drive the evolution of a binary.

We often work with orbital separation (or semimajor axis a), but sometimes it is simply more convenient to use the orbital period ($P_{\rm orb}$). We can convert one to the other using Kepler's third law (Kepler 1609)

$$P_{\rm orb} = 2\pi\left[\frac{a^3}{G(M+m)}\right]^{1/2}. \tag{5.21}$$

Here, G is Newton's constant and the masses are M for the primary and m for the secondary.

The orbital period is probably the shortest timescale in the problem. Binary evolution calculations are often treated in a secular regime. In fact, it is often the thermal timescale of the most massive most evolved star (typically the donor) that sets the timescale on which binary evolution proceeds.

5.3.3 Roche Lobe

The Roche lobe is named after Roche's description of the equipotential surface of two stars in a circular orbit. The shape is symmetric for rotation along the axis connecting the two stars (Paczyński 1971). In Section 4.7.4 and Figure 4.14, we presented an example of using an N-body code to calculate the equipotential surfaces of a binary system. This method is not limited to circular orbits, as in the original formulation of the problem. Luckily, Eggleton (1983) wrote an elegant one page paper on a fitting formula calculating the radius of a sphere with approximately the same volume within the zero-equipotential surface of a star with mass M orbiting a companion of mass m in a circular orbit with semimajor axis a:

$$r_L \simeq \frac{0.49a}{0.5 + q^{-2/3}\ln(1 + q^{1/3})}, \tag{5.22}$$

where $q = m/M$.

When a star of radius R exceeds its Roche radius ($R > r_L$), it starts to transfer mass to its companion, the consequences of which can be quite profound and lead to a wealth of curious phenomena.

5.3.4 Stable Mass Transfer

When one of the stars fills its Roche lobe, mass may start to flow from its surface to the companion (accreting) star. If mass transfer proceeds stably, the variation in

orbital parameters is then calculated using Equation (5.18), otherwise we use the common-envelope prescription Equation (5.23).

The choice of what equation to use is generally a Boolean in binary population synthesis studies, although hybrid approaches have been seen. The main problem with mass transfer in binaries is the number of free parameters associated with the process. In our simple description of the process, we already encountered four such hard to constrain parameters: α, γ, η, and n_j. Each of these parameters is dimensionless, has a value probably of order unity, and is barely constrained within an order of magnitude. These parameters may depend on other parameters in the system, such as the stellar masses, orbital separation, and even on internal stellar parameters. One of the main scientific problems in constraining and determining these parameters hides in the complexities of their coupled effect, and the lack of our ability to perform laboratory experiments. As a professional guess, it may be best to take unity for each of these parameters, see what happens, and then vary them individually, or to stay away from binary evolution population synthesis altogether. The last option is probably scientifically the most honest one.

5.3.5 Common-envelope Evolution

One of the most dramatic events that can follow Roche-lobe overflow is common-envelope evolution. We could spend an entire chapter (maybe even a book) on this topic and all the associated folklore, its relative importance, and the many solutions a plethora of researchers have come up with. The common envelope may be the most controversial topic in modern astrophysics.

We limit our discussion here to a single important question, the degree to which the post-common-envelope binary shrinks while expelling the envelope of the Roche-lobe filling (donor) star. Here, we assume the donor to be the most massive of the two, and be composed of a core (with mass M_c) and an envelope (M_e). The envelope is gravitationally bound to the core, and can be lifted if sufficient energy is deposited in it. Where and how this energy is deposited in the envelope is parameterized with a free parameter α. The internal structure and binding energy of the envelope is parameterized with the free parameter λ. By equating the binding energy of the Roche-lobe filling star's envelope with that of the binary before and after the common envelope, we arrive at the following equation (Webbink 1984):

$$\frac{a}{a_0} = \frac{M_c}{M}\left(\frac{1 + 2a_0}{\alpha\lambda R}\frac{M_e}{m}\right)^{-1}, \tag{5.23}$$

which can be used to calculate the post-common-envelope orbital separation a from the initial orbital separation a_0. The attractive aspect of this equation is that all our ignorance is hidden in the two products $\alpha\lambda$. Decoupling α and λ is practically difficult, but probably quite essential. For a review of this topic, we refer to Ivanova et al. (2013).

5.3.6 Supernova Explosions

A supernova explosion of one of the stars in a binary system has two effects: 1) the exploding star loses mass suddenly, and 2) the resulting compact object may receive

a velocity kick. For clarity and simplicity, we assume that the mass loss is instantaneous compared to the orbital period, after which calculating the orbital elements of the postsupernova binary is a better of solving the Kepler equation.

Here, we do not present the derivation, but just the result (Hills 1983)

$$\frac{a}{a_0} = \left(1 - \frac{\Delta m}{M_0}\right)\left[1 - \frac{2a_0}{r}\frac{\Delta m}{M_0} - 2\frac{v_0 v_k}{v_{c,0}^2}\cos(\theta) - \left(\frac{v_k}{v_{c,0}}\right)^2\right]^{-1}. \tag{5.24}$$

The eccentricity after the supernova then becomes

$$e^2 = 1 - \frac{a_0(1-e^2)^2}{1(a + e_0\cos(\theta_0))^2}\frac{\left(v_{0,x}^2 + v_{k,x}^2 + v_{k,y}^2 + 2v_{0,x}^2 v_{k,x}^2\right)}{G(M_0 - \Delta m)}. \tag{5.25}$$

Here, the angles ϕ and θ represent the kick velocity vector from the companion star in the orbital plane, and perpendicular to the orbital plane. A derivation for arbitrary hierarchy was derived in Pijloo et al. (2012).

5.3.7 Magnetic Braking

Low-mass stars may have magnetically coupled winds, which cause a relatively large loss of angular momentum, even though only a little mass is lost (Rappaport et al. 1983). The amount of angular momentum lost by magnetic braking $\ddot{J}_{\rm mb}$ in terms of the binary's angular momentum J_b can be written in terms of the radii and masses of the two stars.

$$\frac{\ddot{J}_{\rm mb}}{J_b} = \frac{3.8 \times 10^{-30} R_s^{4-\gamma}(M+m)r^{\gamma}\omega^2}{Ma^2}. \tag{5.26}$$

Here, R_s is the radius of the magnetic (primary) star in units of $\mathrm{R}_\odot$, and $\gamma \simeq 2.5$. Usually, magnetic braking is applied to white dwarfs with a main-sequence companion between 0.6 and 1.5 $\mathrm{M}_\odot$.

5.3.8 Gravitational-wave Radiation

General relativistic effects become important for binary evolution once two massive compact object find themselves in a tight orbit. The orbit shrinks at a rate of (Peters 1964)

$$\frac{da_{\rm gr}}{dt} = -\frac{64}{5}\frac{G^3 Mm(M+m)}{c^5 a^3(1-e^2)^{7/2}} f_a, \tag{5.27}$$

with the Taylor-series expansion of the Einstein-Infeld-Hoffman equation (Einstein et al. 1938) to second order in the eccentricity

$$f_a = 1 + \frac{73}{24}e^2 + \frac{37}{96}e^4 + \mathcal{O}(e^6). \tag{5.28}$$

In addition, the circularization of the orbit may be presented in terms of a Taylor-series expansion in e

$$\frac{de_{\rm gr}}{dt} = -\frac{304}{15} e \frac{G^3 M m (M+m)}{c^5 a^4 (1-e^2)^{(5/2)}} f_e, \tag{5.29}$$

with

$$f_e = 1 + \frac{121}{304} e^2. \tag{5.30}$$

5.3.9 Initializing and Evolving a Binary

In AMUSE, these equations for binary evolution are implemented in some form in each of the stellar-evolution codes; these codes are suitable for evolving binaries.

To initialize a binary, one must first initialize the individual stars, and then make a new particle set containing the binary properties, semimajor axis, and eccentricity. The binary particle is then given a pointer to each of the component stars. The current stellar-evolution codes are all binary-aware, but the implementation in the Henyey stellar-evolution codes differ slightly from those in the parameterized codes. Code examples 5.6 and 5.7 present a simple example of how to create and evolve a binary in AMUSE, and how to create a time series of orbital parameters.

To evolve a binary, we often ignore the inclination, argument of pericenter, line of the ascending node, and the mean anomaly. This effectively means that we are studying the evolution of an orbit-averaged system. As a consequence, one cannot trivially couple these binary evolution codes to a dynamical N-body code. To achieve such a coupling, as discussed in Section 8.7.2, one has to assume that binary evolution proceeds on discrete steps comprising multiple orbits during which the orbital phase remains constant. This sounds rather complex, and it is clear what one ideally should do, but it often works in practice. We are also unsure if there is a right way to evolve a binary (or multiple) including stellar physics and gravitational dynamics, except if one would solve all the differential equations systematically and

```
15 def create_double_star(mass_primary, mass_secondary, semi_major_axis, eccentricity):
16     primary_stars = Particles(mass=mass_primary)
17     secondary_stars = Particles(mass=mass_secondary)
18
19     stars = Particles()
20     primary_stars = stars.add_particles(primary_stars)
21     secondary_stars = stars.add_particles(secondary_stars)
22
23     double_star = Particles(semi_major_axis=semi_major_axis, eccentricity=
       eccentricity)
24     double_star.child1 = list(primary_stars)
25     double_star.child2 = list(secondary_stars)
26     return double_star, stars
```

Code example 5.6: Routine to initialize a single binary star, as called in Code example 5.7.

```
def evolve_double_star(
    mass_primary,
    mass_secondary,
    semi_major_axis,
    eccentricity,
    time_end,
    number_of_steps,
):
    double_star, stars = create_double_star(
        mass_primary, mass_secondary, semi_major_axis, eccentricity
    )
    time = 0 | units.Myr
    time_step = time_end / number_of_steps

    code = Seba()
    code.particles.add_particles(stars)
    code.binaries.add_particles(double_star)

    channel_from_code_to_model_for_binaries = code.binaries.new_channel_to(
    double_star)

    t = [] | units.Myr
    a = [] | units.RSun
    e = []
    while time < time_end:
        time += time_step
        code.evolve_model(time)
        channel_from_code_to_model_for_binaries.copy()
        t.append(time)
        a.append(double_star[0].semi_major_axis)
        e.append(double_star[0].eccentricity)
    code.stop()
    return t, a, e
```

Code example 5.7: Routine to evolve a single binary star with `mass_primary`, `mass_secondary`, `semi_major_axis`, and `eccentricity`. The full script can be found in `amuse.examples.binary_evolution`. It should run in a few seconds on a laptop.

synchronously. In principle, it is possible to do this in AMUSE, but the code would be very slow, and without any guarantee that a valid answer would be obtained.

5.3.10 Initial Binary Fraction

One interesting parameter is the fraction of binaries one expects in a system. If you study binaries only, the most common assumption is that all stars are member of a binary system. At a later stage, you can then simply sprinkle in a population of evolved single stars to match the observed statistics. For cluster simulations, however, the binary fraction is important from the beginning.

The fraction of binaries among a certain primary mass seems to increase with mass: more massive stars are more likely to have a bound companion compared to lower-mass stars. We calculate the binary fraction among ZAMS stars from the following equation

$$f_{\rm bin}(M) = \sqrt{R_0}\, M/m_{\rm max}, \tag{5.31}$$

where R_0 is a random number uniformly distributed between 0 and 1, M is the mass of the primary stars (see Section 5.3.11), and $m_{\rm max}$ is the maximum mass in the selected mass function. One could interpret $f_{\rm bin}(M)$ as a probability for binarity for a population with a primary mass close to M, or treat it as an individual probability of the star having a binary companion.

5.3.11 Initial Distribution Functions for Binaries

Binaries in parameterized evolution codes are initialized at birth with the masses of the two stars (M and m, or $q \equiv m/M$), the orbital separation (a), and eccentricity (e). The parameters for an individual binary are randomly taken from distribution functions for each of these parameters. We do not know the shape of these distribution functions from first principles, and observed binary populations are riddled with selection effects.

To randomly select initial conditions for binary population synthesis using the distribution functions, we recommend following these steps:

1. Select the primary mass M from an initial mass function (see Section 3.3).
2. Select the orbital period based on the primary mass-dependent distribution. For example, we could use the function provided in Moe & Di Stefano (2017). This distribution peaks at different periods for different primary masses For solar-type stars, it peaks at $\log(P/\text{days}) \simeq 5$ at about $a \simeq 50$ au.
3. Select the mass ratio ($q \equiv m/M$) based on the primary mass and orbital period using the prescription given by Moe & Di Stefano (2017). This distribution has three components: low-mass primary stars in short orbits tend to have small companion masses, whereas high-mass stars tend to have massive companions, with $q \to 1$ for short-period high-mass stars.
4. The eccentricity (e) is generally distributed thermally. The observed excess of circular orbits for short-period binaries may be a natural consequence of tidal circularization.
5. Finally, the binary fraction is not uniform across all masses, but low for low-mass primary stars and approaching unity for high-mass primaries. For M-dwarfs ($M \sim 0.08$ to $0.60\,\mathrm{M}_\odot$), one could adopt values from Winters et al. (2019). For clarity, we fit a simple power law to the observed binary fractions, of the form

$$f_{\rm binary} = AM^{\alpha}. \tag{5.32}$$

Here, $A = 0.46 \pm 0.03$ and $\alpha = 0.22 \pm 0.03$ with a fit reliability $R^2 \simeq 0.96$. Of course, a binary fraction is capped between 0 and 1.

We recommend implementing these relations using rejection sampling or inverse transform sampling to generate random values from these distributions (Devroye 1986). The minimum orbital period is set to correspond to Roche-lobe filling on the ZAMS.

By following these steps and using the provided distribution functions, you can generate a reasonable set of initial conditions for binary population synthesis that is statistically consistent with the observed characteristics of binary star systems.

5.3.12 Rejuvenation due to Collision or Accretion

When a star accretes material, by mass transfer from a companion or following a collision, a number of important changes occur. It starts to burn its nuclear fuel at a higher rate, and becomes hotter and brighter. Possibly the most important change, however, is the possibility that some of the accreted material finds its way into the stellar core, where nuclear fusion occurs. As a result, the accreting star may appear younger than the surrounding stars in a stellar cluster. Such "rejuvenated" stars are called *blue stragglers* (see Section 5.4.2). We will discuss the physics of stellar collisions in more detail in Section 6.4.2.

Rejuvenation is an important aspect of stellar and binary evolution, but different codes adopt very different approaches to the process. A Henyey code manages accretion onto a star by adding fresh gas to the outermost shell in the one-dimensional stellar structure. The internal solvers for the structure equations of the star will then resolve this addition and arrive at a new structure with the appropriate burning rates, etc., and ultimately mix the new gas into the interior. In a parameterized code, stars evolve along precalculated tracks and new tracks cannot easily be generated. Therefore, these codes incorporate an approximate prescription for on what track, and where on that track, an accreting star should reappear. This, of course, is a complete fudge, but it works reasonably well in practice. We warn the reader that these prescriptions are chosen because they seem reasonable and often work, but they are generally not tested against detailed calculations.

The two parameterized codes in AMUSE employ similar approaches toward rejuvenation, although they differ in detail. Following accretion, the star in `SSE` is restarted on the ZAMS appropriate to the new mass. In `SeBa`, the evolution of a star is separated into partial tracks that represent the different stellar-evolution phases (see Table A.1). After accretion, a star is placed at the same relative position between the beginning and end of the same track of a star with the increased mass. The relative age of the star along this track is reduced according to the fraction of accreted mass relative to the original mass of the star. In this way, a star can accrete small amounts of mass in consecutive phases of accretion.

For example, suppose that a 5 $M_\odot$ star halfway through the main sequence stage —at an age of 0.5 $t_{\rm tams}(5\ M_\odot)$—accretes 0.1 $M_\odot$ following a phase of rapid mass transfer. Here, $t_{\rm tams}$ refers to the *terminal-age main sequence*, the point where the star moves from the main sequence to the subgiant branch on the H–R diagram. According to the prescription in `SSE`, this new 5.1 $M_\odot$ star will start its evolution as a zero-age star at this mass. In `SeBa`, this same star will be restarted at a fraction of $(0.5\ M_\odot – 0.1\ M_\odot\ /5\ M_\odot) \times t_{\rm tams}(5.1\ M_\odot) = 0.48\ t_{\rm tams}(5.1 M_\odot)$.

A nice aspect of parameterized stellar-evolution codes is their predictable behavior. The Henyey codes behave less predictably, but they provide a better, if more expensive, answer to this complicated problem.

5.3.13 Reading and Writing Binary Evolution Files

The data structure for stellar binaries (and higher-order hierarchies) is somewhat more elaborate than for a single particle set. Storing stellar multiples is therefore also somewhat more elaborate. Writing stellar binaries can be realized by separately storing the individual stars (here called `stars`) and the binaries (here called `binaries`). We write them in two separate files.

```
write_set_to_file(stars, "individual_tars.hdf5", "amuse")
write_set_to_file(binaries, "stars_binary_pairs.hdf5", "amuse")
```

We can read the data from the duplicate files with the reverse operation,

```
stars = read_set_from_file('individual_stars.hdf5','amuse')
binaries = read_set_from_file('stars_binary_pairs.hdf5','amuse')
```

5.4 Examples

5.4.1 Response of a Star to Mass Loss

An important issue in stellar evolution is the response of a star to changes in its mass. This response is generally expressed as the dimensionless adiabatic or the thermal (Kelvin–Helmholtz) response, $\zeta_{\rm ad}$ or $\zeta_{\rm th}$, where we define, for mass loss dm

$$\zeta = \frac{d \ln r}{d \ln m} \tag{5.33}$$

(Hjellming & Webbink 1987), where $r(m)$ is the radius in the star containing mass m and the expression is evaluated at the surface of the star (where $m = M$). The response ζ can be compared with the change in the size of the Roche lobe in order to determine the rate at which the star loses mass due to Roche-lobe overflow (Passy et al. 2012).

In binary population synthesis codes, one often adopts a simple parameterization of ζ—it is generally held constant, independent of dm/dt, mass, or the evolutionary stage (Ge et al. 2010). In `SeBa`, for example, $\zeta_{\rm th} = 0$ for main-sequence stars less massive than 0.4 $M_\odot$, $\zeta_{\rm th} = 0.55$ for stars more massive than 1.5 $M_\odot$, and $\zeta_{\rm th} = 0$ for all other stars, except for those in the Hertzsprung gap, for which $\zeta_{\rm th} = -2$, independent of mass. The negative sign here indicates that the star shrinks as a result of mass loss. In these parameterizations, $\zeta_{\rm th}$ is assumed to be independent of dm/dt, which is not realistic because the response of the star to mass loss must depend on the mass-loss rate.

In AMUSE, we can run a direct stellar-evolution code, `EVTwin` or `MESA`, to calculate ζ and also study the dependence of ζ on the rate of mass loss. Code example 5.8 presents a simple routine to calculate the ζ coefficients for a particular star. This function is called by the main routine in Code example 5.9, in which a star of initial mass M is evolved. Figure 5.9 presents the evolution of ζ as a function of

```
def calculate_zeta(star, metallicity, dmdt):
    stellar = Mesa()
    stellar.parameters.metallicity = metallicity
    stellar.particles.add_particles(star)
    radius_old = star.radius
    star.mass_change = dmdt
    mass_step = 0.01 * star.mass
    star.time_step = mass_step / dmdt
    stellar.particles.evolve_one_step()
    radius_new = stellar.particles[0].radius
    dlnr = (radius_new - radius_old) / radius_old
    dlnm = (stellar.particles[0].mass - star.mass) / star.mass
    zeta = dlnr / dlnm
    stellar.stop()
    return zeta
```

Code example 5.8: Routine to calculate $\zeta_{\rm th}$ for a star. This function is called by the main function Code example 5.9.

```
def massloss_response(
    mass=1.0 | units.MSun, metallicity=0.02, dmdt=-0.01 | units.MSun / units.yr
):
    stellar = Mesa()
    stellar.parameters.metallicity = metallicity
    bodies = Particles(mass=mass)
    stellar.particles.add_particles(bodies)
    stellar = turnon_massloss(stellar)
    channel_to_framework = stellar.particles.new_channel_to(bodies)
    copy_argument = ["age", "mass", "radius", "stellar_type"]
    while stellar.particles[0].stellar_type < Second_Asymptotic_Giant_Branch:
        stellar.particles.evolve_one_step()
        channel_to_framework.copy_attributes(copy_argument)
        star = stellar.particles.copy()
        zeta = calculate_zeta(star, metallicity, dmdt)
        print(
            "Zeta=",
            zeta[0],
            bodies[0].age,
            bodies[0].mass,
            bodies[0].radius,
            dmdt,
            bodies[0].stellar_type,
        )
    stellar.stop()
```

Code example 5.9: Main function to evolve a single star of mass M in time. Here, we use the internal stellar-evolution code parameter `Second_Asymptotic_Giant_Branch = 6 | units.stellar_type`, which is implemented as a part of the `units` package. This function calls Code example 5.8 to calculate ζ. The full script can be found in `amuse.examples.massloss_response`.

time for a few stars and mass-loss rates. We used MESA in these cases and continued the calculations until the code became impractically slow, which generally happened somewhere on the first AGB (see Figure 5.3).

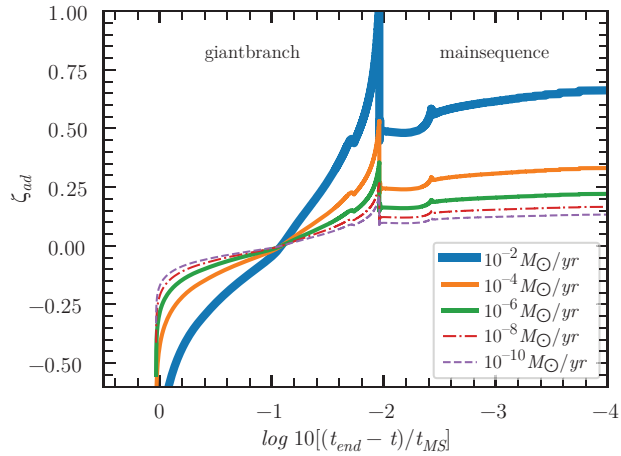

Figure 5.9. Response of an $M = 1\ \mathrm{M}_\odot$ star to mass loss, expressed as a function of time (relative to the main sequence lifetime). We performed the calculation for mass-loss rates $dm/dt = 10^{-2}\,\mathrm{M}_\odot\,\mathrm{yr}^{-1}$ (top curve), $dm/dt = 10^{-4}\,\mathrm{M}_\odot\,\mathrm{yr}^{-1}$, $10^{-6}\,\mathrm{M}_\odot\,\mathrm{yr}^{-1}$, $10^{-8}\,\mathrm{M}_\odot\,\mathrm{yr}^{-1}$, and $10^{-10}\,\mathrm{M}_\odot\,\mathrm{yr}^{-1}$ (bottom curve). This figure was created using a script based on `amuse.examples.stellar.massloss_response`. The calculation took several hours on a laptop, so we include the data file (`Zeta_M1MSun_for_various_dmdt.data` in the `data` directory) for a $1\ \mathrm{M}_\odot$ star and plot this using the `amuse.examples.plot_stellar_massloss_response` module.

5.4.2 Blue Stragglers in M67

M67 was discovered between 1772 and 1779 by Johann Gottfried Koehler, who described it as a "rather conspicuous nebula in elongated figure, near Alpha of Cancer." Charles Messier described the cluster on 1780 April 6 as a "cluster of small stars with nebulosity below the southern claw of the Crab." He gave the cluster its current identification M67. In 1783, William Herschel counted a total of 20 stars. Today, M67 is known to host nearly 1900 stars (Uribe et al. 2007) and goes under many names, among them HR3515, OCl549.0, C0847 + 120, NGC2682, and [KPR2004b]212.

The age of a cluster is often measured by the distribution of stars in the H–R diagram. Key to this age determination is the main sequence turn-off—the temperature and luminosity at which stars leave the relatively hot main sequence and start to cross the Hertzsprung gap to cooler regions. This turn-off point is associated with a certain stellar mass and hence age. Stars more massive than the turn-off mass have already started to cross the Hertzsprung gap, whereas lower-mass stars are still on the main sequence.

For the cluster M67, the turn-off mass is $\sim$1.4 $\mathrm{M}_\odot$ (Boffin et al. 2015, p.51). There is also a population of stars that lie along the main sequence, but are brighter and hotter than the main sequence turn-off. These stars are called blue stragglers, and were first discovered in the globular cluster M3 (Sandage 1953). Blue stragglers are commonly visible in any star cluster for which a turn-off can be clearly determined, but their origin remains somewhat elusive. Table 5.3 lists some fundamental and derived parameters for five blue stragglers in M67. Their temperatures and

Table 5.3. Observed Parameters for Some M67 Blue Stragglers

name	$T_{\rm eff}$ [K]	V mag	L [$L_\odot$]	$\log g$
S984(F134)	6170	12.259	10.55	3.9
S975	6820	11.078	3.558	4.4
S997	6675	12.127	9.350	4.4
S1082	7050	11.226	4.078	4.5
S2204	6650	12.892	18.91	4.6

Notes. M67 is about 4 Gyr old (Shetrone & Sandquist 2000). The luminosity is calculated from $10^{(V-\beta)/2.5}$. Here, $\beta = V - M_v = 9.7$ is the distance modulus.

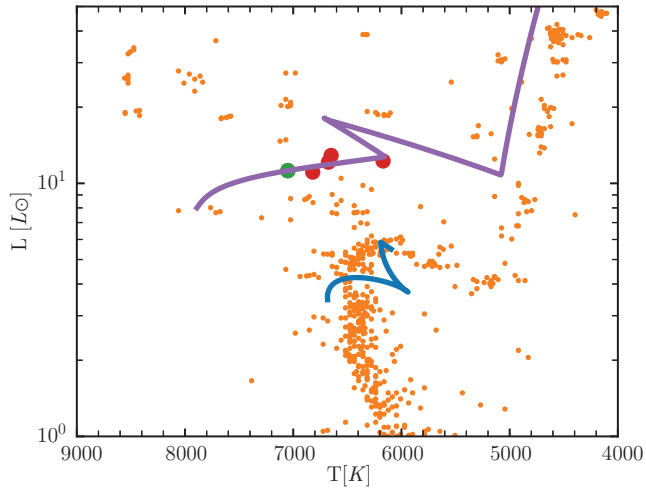

Figure 5.10. H–R diagram for the star cluster M67. The data are from http://webda.physics.muni.cz/ (Eggen & Sandage 1964). The red (the stars S984, S975, S997, and S2204) and green (S1082) bullets are the blue stragglers listed in Table 5.3. The other bullet points (orange) are the M67 stars. Two lines indicate the evolutionary track for a star near the turn-off of 1.4 $M_\odot$ (blue) and 1.7 $M_\odot$ (purple), both at twice the solar metallicity ($Z = 0.02$). This figure was generated using the script `amuse.examples.plot_M67withBSS_tracks`. It should take 5–10 s to run on a laptop.

luminosities can be compared directly with the output of any stellar-evolution module in AMUSE.

Because they are brighter and hotter than the turn-off, it has been suggested that blue stragglers result from stellar mergers during binary evolution or following stellar collisions in the dense central region of star clusters. Their higher masses and younger ages are then direct consequences of their collisional history. Here, we discuss their possible association with stellar evolution; we will return to the possibility of a collisional origin in Section 8.5.2.

Figure 5.10 presents a historical H–R diagram for M67, including the five blue stragglers listed in Table 5.3. We overplot two evolutionary tracks to demonstrate how the turn-off stars evolve to an age of 4 Gyr. The upper 1.7 $M_\odot$ evolutionary track runs

nicely through the blue stragglers, but the tip of the curve is off the plot. This indicates that these stars would already have evolved to the white-dwarf stage, so they would be absent in this observation if they really were 4 Gyr old. The other track (blue curve) gives the evolutionary track of a 1.4 $M_\odot$ star to the same age. These stars are still visible in the cluster near the end of the turn-off or early Hertzsprung gap.

5.5 Validation

Stellar-evolution codes are hard to validate. One major problem is that we cannot observe the full evolution of any star. Generally, there are two observable parameters that can be tested directly—the effective temperature (or something equivalent) and the luminosity. Recent advances in observational techniques and the discovery of planets orbiting many stars have given us new insights into stellar atmospheres, pulsations, etc., but these are often not solved for in stellar-evolution codes (however, see also Christensen-Dalsgaard et al. 1996; Aerts et al. 2010).

The equations of stellar structure and evolution are generally quite well-behaved. There are few significant nonlinear effects, and there is no fundamental reason to think that solving the various differential equations should not provide the right answer (mathematically speaking), so long as the star is in thermal and dynamical equilibrium. Subsequent validation is carried out by comparing stellar-evolution tracks with star cluster isochrones, or even with the positions of individual stars in the H–R diagram. This is the basic reason why stellar-evolution theory has been so successful.

The only star for which a more thorough validation can be carried out is the Sun, because its surface parameters are quite well determined. In Figure 5.11, we compare the solar temperature and luminosity with the results of the four stellar-evolution codes currently available in AMUSE. Although the differences are not large, they are still significant, particularly considering the small error bars on the observed values, which are smaller than the plotted symbols (see Williams 2013).

It remains challenging to compare individual stellar-evolution codes, particularly if they give slightly different results for the same input parameters (mass, metallicity, and age). Often, the differences are fairly small and are not a major concern for many coupled problems that involve hydrodynamics, radiative transport, or gravitational dynamics. The global mass loss of a population of stars in the central portion of a star cluster is the result of a complicated combination of dynamical and stellar evolution, and may depend only to first order on the specific stellar-evolution code used. It remains important, however, to be aware of these small differences and recognize that a certain subspace of initial parameters may give different results, depending on the adopted stellar-evolution code.

5.6 Assignments

5.6.1 Stellar Comparison

Rewrite the simple script `plot_solar_comparison` used in Figure 5.11 to compare the computed temperatures and luminosities of some other stars. The best stars to try this on are the closest and brightest—Rigel, Betelgeuse (see

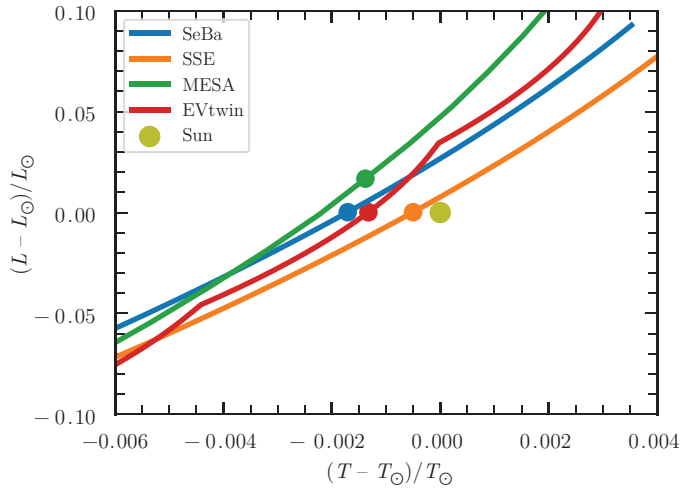

Figure 5.11. Comparison of the effective temperature and luminosity for the Sun, calculated using the four stellar-evolution codes in AMUSE. The calculations were performed for a star with solar metallicity and an initial mass of 1 $M_\odot$, evolved from 4 Gyr to 5.2 Gyr (curves from left to right). The points indicate the moments (using each code's internal time resolution) when the Sun is best represented by the models. These occurred at 5.18 Gyr, 5.20 Gyr, 4.33 Gyr, and 4.30 Gyr for `SeBa`, `SSE`, `MESA`, and `EVTwin`, respectively. The solar temperature and luminosity are taken from Allen's Astronomical Quantities (Cox 2000). This figure was generated using the script `amuse.examples.plot_solar_comparison`. It should take about 20 s to run on a laptop.

Figure 5.1), the red supergiant R Sculptoris, and both stars in the Spica binary system (see Table 5.4). Note that, even for these stars, the direct stellar-evolution codes may be unable to converge to a solution, in which case they will simply break.

5.6.2 Ages of the M67 Blue Stragglers

The five blue stragglers in M67 listed in Table 5.3 show a remarkable coherence in terms of their mass. In Section 5.4.2 we demonstrated that each of them lies close to the main sequence location of a 1.7 $M_\odot$ star of twice solar metallicity. Their ages, however, are clearly not the same because each of them is positioned differently along the 1.7 $M_\odot$ evolutionary track.

Using any of the stellar-evolution codes in AMUSE, determine the ages of each of the blue stragglers in Table 5.3, and perhaps even improve on the listed mass estimate, assuming that they were formed as normal ZAMS stars.

5.6.3 Constructing Isochrones

Write an AMUSE script to construct a series of 10 isochrones equally spaced in time from 1 Gyr to 10 Gyr for stars of 0.2 $M_\odot$ to 20 $M_\odot$, with solar metallicity. Plot the isochrones in the temperature (K)–luminosity ($L_\odot$) plane.

Table 5.4. Observed and Measured Parameters for Some Nearby Bright Stars

Name	Ref	Mass [$M_\odot$]	Age [Myr]	T [K]	L [$L_\odot$]	R [$R_\odot$]	[Fe/H]
Rigel	1	21 ± 3	8 ± 1	12, 100	$1.0–1.40 \times 10^5$	78.9 ± 7.4	−0.06
Spica A	2	11.4 ± 1.2	12.5	22,400	12,100	7.40 ± 0.57	–
Spica B	3	7.21 ± 0.75	12.5	18,500	1,500	3.64 ± 0.28	–
Betelgeuse	4	7.7–20	7.3	3140–3641	$9–15 \times 10^5$	950–1200	0.05
R Sculptoris	5	1.3 ± 0.6	–	2640 ± 80	5500	355 ± 55	–

References 1: Przybilla et al. (2006, 2010); 2: Lyubimkov et al. (1995); Harrington et al. (2009); 3: Palate et al. (2013); Tkachenko et al. (2016); 4: Ramírez et al. (2000); Mohamed et al. (2012); 5: Wittkowski et al. (2017).

Convert the H–R diagram into a color–magnitude diagram. Luminosity can be converted to magnitudes in the usual way, by taking distance modulus and interstellar reddening [$E(B - V) = 0.04$] (Taylor 2007) into account,

$$m_{\rm star} = m_{\odot} - 2.5 \log \left[\frac{L_{\rm star}}{\rm L_{\odot}} \left(\frac{\rm pc}{d_{\rm star}} \right)^2 \right]. \tag{5.34}$$

Here, the apparent magnitude of the Sun is $m_{\odot} = -26.73$, and its distance is $d_{\odot} \simeq 2.25 \times 10^{-8}$ pc (Allen 1955; Cox 2000). Color $(B - V)$ can be converted to temperature using the following polynomial fit:

$$\log T(K) \simeq [14.551 - (B - V)]/3.684. \tag{5.35}$$

Address the following questions:

- What is the best stellar-evolution module to use for this task?
- What age best fits the observed star cluster M67? The data for M67 can be found at http://www.univie.ac.at/webda/cgi-bin/ocl_page.cgi?cluster=M67.
- What distance do you have to assume for the best fit?
- What age or metallicity spread do you need in order to obtain a better comparison between the simulated cluster and the observations?
- What additional parameters could you introduce in order to improve the fit?

References

Aerts, C., Christensen-Dalsgaard, J., & Kurtz, D. W. 2010, Asteroseismology (Berlin: Springer)

Allen, C. W. 1955, Astrophysical Quantities (London: Athlone Press)

Bazán, G., Dearborn, D. S. P., Dossa, D. D., et al. 2003, Conf. Ser. 293, 3D Stellar EvolutionASP ed. S. Turcotte, S. C. Keller, & R. M. Cavallo (San Francisco, CA: ASP) 1

Bethe, H. A. 1939, PhRv, 55, 434

Boffin, H. M. J., Carraro, G., & Beccari, G. 2015, Ecology of Blue Straggler Stars (Berlin: Springer)

Bradt, H. 2014, Astrophysics Processes (Cambridge: Cambridge Univ. Press)

Brott, I., de Mink, S. E., Cantiello, M., et al. 2011, A&A, 530, A115

Chiu, H. Y. 1964, AnPhy, 26, 364

Christensen-Dalsgaard, J., Dappen, W., Ajukov, S. V., et al. 1996, Sci., 272, 1286

Clapeyron, É 1834, Mémoire sur la puissance motrice de la chaleur (Paris: Jacques Gabay)

Clausius, R. 1857, AnP, 176, 353

Cox, A. N. 2000, Allen's Astrophysical Quantities (New York: AIP Press)

Devroye, L. 1986, Non-uniform Random Variate Generation (New York: Springer)

Dolan, M. M., Mathews, G. J., Lam, D. D., et al. 2016, ApJ, 819, 7

Duquennoy, A., Mayor, M., & Mermilliod, J. C. 1992, Binaries as Tracers of Star Formation, ed. A. Duquennoy, & M. Mayor (Cambridge: Cambridge Univ. Press) 52

Eggen, O. J., & Sandage, A. R. 1964, ApJ, 140, 130

Eggleton, P. P. 1983, ApJ, 268, 368

Eggleton, P. P., Tout, C., Pols, O., et al. 2011, STARS: A Stellar Evolution Code, Astrophysics Source Code Library, ascl:1107008

Eggleton, P. 2006, Evolutionary Processes in Binary and Multiple Stars (Cambridge: Cambridge Univ. Press)
Eggleton, P. P. 1971, MNRAS, 151, 351
Eggleton, P. P., Fitchett, M. J., & Tout, C. A. 1989, ApJ, 347, 998
Einstein, A., Infeld, L., & Hoffmann, B. 1938, AnMat, 39, 65
Ekström, S., Georgy, C., Eggenberger, P., et al. 2012, A&A, 537, A146
Gaburov, E., Lombardi, J. C., & Portegies Zwart, S. 2008, MNRAS, 383, L5
Ge, H., Hjellming, M. S., Webbink, R. F., Chen, X., & Han, Z. 2010, ApJ, 717, 724
Georgy, C., Meynet, G., Walder, R., Folini, D., & Maeder, A. 2009, A&A, 502, 611
Groh, J. H., Ekström, S., Georgy, C., et al. 2019, A&A, 627, A24
Harrington, D., Koenigsberger, G., Moreno, E., & Kuhn, J. 2009, ApJ, 704, 813
Haubois, X., Perrin, G., Lacour, S., et al. 2009, A&A, 508, 923
Hayashi, C. 1961, PASJ, 13, 450
Helfand, D. J., & Tademaru, E. 1977, ApJ, 216, 842
Henyey, L. G., Wilets, L., Böhm, K. H., Lelevier, R., & Levee, R. D. 1959, ApJ, 129, 628
Henyey, L. G., Forbes, J. E., & Gould, N. L. 1964, ApJ, 139, 306
Hills, J. G. 1983, ApJ, 267, 322
Hjellming, M. S., & Webbink, R. F. 1987, ApJ, 318, 794
Hurley, J. R., Pols, O. R., & Tout, C. A. 2000, MNRAS, 315, 543
Hut, P., Shara, M. M., Aarseth, S. J., et al. 2003, NewA, 8, 337
Ivanova, N., Justham, S., Chen, X., et al. 2013, A&ARv, 21, 59
Kapil, V., Mandel, I., Berti, E., & Müller, B. 2023, MNRAS, 519, 5893
Kepler, J. 1609, Astronomia Nova Vol. 1 (Heidelberg: G. Voegelinus)
Kingdon, K. H., & Langmuir, I. 1923, PhRv, 22, 148
Kippenhahn, R., & Weigert, A. 1967, ZA, 65, 251
Köhler, K., Langer, N., de Koter, A., et al. 2015, A&A, 573, A71
Langer, N. 2012, ARA&A, 50, 107
Laplace, E., Götberg, Y., de Mink, S. E., Justham, S., & Farmer, R. 2020, A&A, 637, A6
Livio, M., & Warner, B. 1984, Obs, 104, 152
Lombardi, J. C., Thrall, A. P., Deneva, J. S., Fleming, S. W., & Grabowski, P. E. 2003, MNRAS, 345, 762
Lombardi, J. C., Warren, J. S., Rasio, F. A., Sills, A., & Warren, A. R. 2002, ApJ, 568, 939
Lyne, A. G., & Lorimer, D. R. 1994, Natur, 369, 127
Lyubimkov, L. S., Rachkovskaya, T. M., Rostopchin, S. I., & Tarasov, A. E. 1995, ARep, 39, 186
Maeder, A., & Meynet, G. 1988, A&AS, 76, 411
Meynet, G., & Maeder, A. 2005, A&A, 429, 581
Meynet, G., Maeder, A., Schaller, G., Schaerer, D., & Charbonnel, C. 1994, A&AS, 103, 97
Moe, M., & Di Stefano, R. 2017, ApJS, 230, 15
Mohamed, S., Mackey, J., & Langer, N. 2012, A&A, 541, A1
Ohnaka, K., Weigelt, G., & Hofmann, K. H. 2017, Natur, 548, 310
Paczyński, B. 1971, ARA&A, 9, 183
Palate, M., Koenigsberger, G., Rauw, G., Harrington, D., & Moreno, E. 2013, A&A, 556, A49
Passy, J.-C., Herwig, F., & Paxton, B. 2012, ApJ, 760, 90
Paxton, B., Bildsten, L., Dotter, A., et al. 2010, Modules for Experiments in Stellar Astrophysics, Astrophysics Source Code Library, ascl:1010.083
Paxton, B., Bildsten, L., Dotter, A., et al. 2011, ApJS, 192, 3

Peters, P. C. 1964, PhRv, 136, 1224
Pijloo, J. T., Caputo, D. P., & Portegies Zwart, S. F. 2012, MNRAS, 424, 2914
Portegies Zwart, S. F., & Verbunt, F. 1996, A&A, 309, 179
Portegies Zwart, S. F., & Verbunt, F. 2012, SeBa: Stellar and Binary Evolution, Astrophysics Source Code Library, ascl:1201.003
Portegies Zwart, S. F., & Yungelson, L. R. 1998, A&A, 332, 173
Przybilla, N., Butler, K., Becker, S. R., & Kudritzki, R. P. 2006, A&A, 445, 1099
Przybilla, N., Firnstein, M., Nieva, M. F., Meynet, G., & Maeder, A. 2010, A&A, 517, A38
Ramírez, S. V., Sellgren, K., Carr, J. S., et al. 2000, ApJ, 537, 205
Rankine, W. J. M. 1853, Proc. of the Philosophical Society of Glasgow, Vol. 3, ed. W. J. Millar (London: Richard Griffin & Co.) 276
Rappaport, S., Verbunt, F., & Joss, P. C. 1983, ApJ, 275, 713
Rizzuti, F., Hirschi, R., Arnett, W. D., et al. 2023, MNRAS, 523, 2317
Saha, M. N. 1921, RSPSA, 99, 135
Salpeter, E. E. 1955, ApJ, 121, 161
Sandage, A. R. 1953, AJ, 58, 61
Schaerer, D., de Koter, A., Schmutz, W., & Maeder, A. 1996, A&A, 310, 837
Shetrone, M. D., & Sandquist, E. L. 2000, AJ, 120, 1913
Smartt, S. J. 2009, ARA&A, 47, 63
Spera, M., Mapelli, M., & Bressan, A. 2015, MNRAS, 451, 4086
Stasińska, G., & Schaerer, D. 1997, A&A, 322, 615
Steig, W. 2008, Shrek! (New York: Square Fish)
Takahashi, K., & Langer, N. 2021, A&A, 646, A19
Taylor, B. J. 2007, AJ, 133, 370
Tkachenko, A., Matthews, J. M., Aerts, C., et al. 2016, MNRAS, 458, 1964
Toonen, S., Nelemans, G., & Portegies Zwart, S. 2012, A&A, 546, A70
Tutukov, A. V., & Fedorova, A. V. 2001, ARep, 45, 882
Uribe, A., Barrera-Rojas, R. S., & Brieva, E. 2007, Conf. Ser. 371, Statistical Challenges in Modern Astronomy IVASP ed. G. J. Babu, & E. D. Feigelson (San Francisco, CA: ASP) 403
Varshavskii, V. I., & Tutukov, A. V. 1975, AZh, 52, 227
Webbink, R. F. 1984, ApJ, 277, 355
Williams, R. D. 2013, Sun Fact Sheet (Washington, DC: NASA)
Winters, J. G., Henry, T. J., Jao, W.-C., et al. 2019, AJ, 157, 216
Wittkowski, M., Hofmann, K. H., Höfner, S., et al. 2017, A&A, 601, A3
Zahn, J. P. 1977, A&A, 57, 383